Indian Institute of Technology Bombay
Department of Computer Science and Engineering

# Profiling and Optimizing LLM Inference at Scale

*Systems for LLMs*




**Saksham Rathi**

Under the guidance of

**Prof. Mythili Vutukuru**

Bachelor's Thesis Project

Academic Year: 2025–26

# Contents

**List of Tables** 4

**List of Figures** 5

**1 Introduction** 7
- 1.1 Motivation . . . 7
- 1.2 Problem Statement . . . 7
- 1.3 Brief Overview of the Work Accomplished . . . 8
- 1.4 Organization of the Report . . . 8

**2 Background** 9
- 2.1 Stages in the Model Creation Pipeline . . . 9
- 2.2 Transformer Architecture . . . 10
  - 2.2.1 Tokenization . . . 10
  - 2.2.2 Embedding and Attention . . . 10
  - 2.2.3 Multi-Head Masked Attention . . . 11
- 2.3 Transformer Based Large Langauge Models . . . 11
  - 2.3.1 Autoregressive Generation . . . 12
- 2.4 LLM Inferencing . . . 13
  - 2.4.1 Forward Pass . . . 13
  - 2.4.2 Token Probability Distribution . . . 13
  - 2.4.3 Generation Loop . . . 14
  - 2.4.4 Modes of Inference . . . 14
- 2.5 Inference Engines . . . 14
  - 2.5.1 Optimizations in Inference Engines . . . 15
- 2.6 Metrics . . . 15
  - 2.6.1 Throughput . . . 15
  - 2.6.2 Latency . . . 15

**3 vLLM** 16
- 3.1 Memory Challenges . . . 17
- 3.2 vLLM . . . 17
  - 3.2.1 PagedAttention . . . 17
  - 3.2.2 KV Cache Manager . . . 17
  - 3.2.3 Parallel Sampling . . . 18
  - 3.2.4 Scheduling and Preemption . . . 18
- 3.3 Automatic Prefix Caching in vLLM . . . 19
  - 3.3.1 Overview . . . 19
  - 3.3.2 System Architecture . . . 19
  - 3.3.3 Core Operations . . . 20

3.4 v1 Scheduler . . . 20

**4 SGLang** **23**
4.1 LM Programs . . . 23
4.2 Programming Model . . . 23
4.3 RadixAttention . . . 24
4.4 Compressed Finite State Machine . . . 25
4.5 FastTree: Optimizing Radix Tree based KV Cache Computation . . . 25

**5 Matrix Multiplication Analysis** **27**
5.1 Motivation . . . 27
5.2 Algorithm . . . 27
5.3 Nsight Systems GPU Metrics . . . 28
5.4 Analysis of GPU Utilization using DCGMI . . . 30

**6 Initial Experiments** **32**
6.1 System Prompt in Cache Experiment . . . 32
6.1.1 Example 1 . . . 32
6.1.2 Example 2 . . . 33
6.1.3 Observations . . . 34
6.1.4 Analysis over different steps and using different metrics . . . 34
6.1.5 Profiling across pipeline stages . . . 35
6.2 Batch of Requests . . . 35

**7 Experiments - Single Family of Requests** **36**
7.1 Throughput and Latency Measurements across different techniques . . . 36
7.2 GPU Memory Breakdown . . . 38
7.3 LLM Request Timelines . . . 38
7.3.1 Chunked Prefill Enabled . . . 40
7.4 Maximum Throughput with vLLM . . . 41
7.5 Throughput vs Load Comparison For vLLM . . . 42
7.6 Compute Utilization across various max batch sizes . . . 43

**8 Experiments - Multiple Family of Requests** **45**
8.1 Introduction . . . 45
8.2 How to generate such a workload? . . . 45
8.3 Families of Requests across various models . . . 45
8.4 Effect of Batch Decomposition on Inter-Scheduler Latency . . . 46
8.5 Merging queries belonging to the same family into a single batch . . . 47
8.6 Effect of Max Batch Size on Throughput for multiple families . . . 49
8.7 Varying all the parameters together . . . 49
8.8 Comparison of vLLM with SGLANG - for multiple families . . . 52
8.9 GPU Utilization Metrics across different family and batch sizes . . . 54

**9 Future Steps** **55**

**A Maths behind LLMs** **56**
A.1 Transformer FLOPs . . . 56
A.1.1 FLOPs Calculation . . . 56
A.1.2 Model parameters (TinyLlama-1.1B-Chat) . . . 57

A.1.3 FLOPs Calculation for TinyLlama-1.1B-Chat . . . . . 57
A.1.4 Efficiency Calculation . . . . . 57
A.1.5 Efficiency Calculation . . . . . 57
A.1.6 Summary of the nomenclature used . . . . . 58

**B Source Code** **59**

**Bibliography** **60**

# List of Tables


5.1 Nsight Systems GPU Metrics . . . . . 29
5.2 DCGMI GPU Metrics . . . . . 30

6.1 Profiler Output . . . . . 34
6.2 Profiling results for various pipeline stages . . . . . 35
6.3 Time taken to process a batch of requests using different techniques . . . . . 35

7.1 GPU Memory Breakdown and KV Cache Usage . . . . . 38

8.1 Throughput with multiple families across various models . . . . . 46
8.2 Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging . . . . . 47
8.3 Throughput across different $k$ levels . . . . . 52
8.4 Throughput comparison (in req/s). . . . . 53

A.1 Variables used in FLOPs . . . . . 58

# List of Figures


2.1 LM Model [3] . . . 9
2.2 Stages in the Model Creation Pipeline [1] . . . 9
2.3 Architecture of a Large Language Model [3] . . . 11
2.4 Prefill vs Decode [10] . . . 13
2.5 Architecture of an Inference Engine [3] . . . 14

3.1 Memory layout when serving an LLM with 13B parameters on NVIDIA A100 [11] 16
3.2 vLLM system overview [11] . . . 17
3.3 Illustration of prefix caching: shared prefixes reuse cached KV blocks. . . . 19
3.4 Internal organization of the KV cache manager. . . . 20

4.1 SGLANG Frontend Language Model [13] . . . 24
4.2 SGLANG Radix Tree Example [13] . . . 25
4.3 KV cache sharing through Radix Tree [12] . . . 26
4.4 Greedy runtime optimization algorithm [12] . . . 26

5.1 NSYS GPU Metrics for a matrix multiplication run . . . 29
5.2 SM Active and SM Occupancy across different matrix sizes . . . 31

7.1 Performance metrics across varying QPS . . . 37
7.2 20 Requests per second . . . 39
7.3 25 Requests per second . . . 39
7.4 All requests at once . . . 40
7.5 Request Timeline with GPU Metrics . . . 40
7.6 Chunked Prefill Enabled - All requests at once . . . 41
7.7 Number of batch outputs - chunked prefill disabled . . . 42
7.8 Number of batch outputs - chunked prefill enabled . . . 42
7.9 Throughput vs Load . . . 43
7.10 Throughput vs Load for different models for heavy workloads . . . 43
7.11 SM Active and SM Occupancy across different max batch sizes . . . 44

8.1 Generation of families by prepending unique identifiers, resulting in separate KV Cache blocks despite shared system prompt content. . . . 46
8.2 Times spent across different batch decompositions . . . 47
8.3 Throughput vs Load across different pre-filling strategies . . . 48
8.4 Throughput vs Load across different numbers of families . . . 48
8.5 Throughput vs Load across different numbers of families . . . 49
8.6 Throughput vs Max Batch Size for multiple families . . . 49
8.7 Visualization of query ordering strategies for 4 families (4 requests each): (0) sorted, (1) chunked random, (2) completely random. Each dashed box represents a batch of 4 requests. . . . 50

8.8 Throughput difference across different batch nature levels . . . . . 50
8.9 Throughput vs Workload by varying other parameters (10 families) . . . . . 50
8.10 Throughput vs Workload by varying other parameters (100 families) . . . . . 51
8.11 Comparison for $k = 500$ and $k = 499$ . . . . . 52
8.12 vLLM vs SGLANG - varying parameters . . . . . 53
8.13 vLLM vs SGLANG - Relative Speedup across randomness levels . . . . . 53
8.14 vLLM vs SGLANG - Throughput levels for a batch size of 500 . . . . . 53
8.15 SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes and max batch sizes . . . . . 54

A.1 FLOPs Calculation Breakdown [9] . . . . . 57

# Chapter 1

# Introduction

The recent surge in the deployment of Large Language Models (LLMs) has transformed a wide spectrum of interactive applications, ranging from conversational agents and enterprise assistants to data analysis tools and reasoning systems. Unlike traditional inference workloads—such as image classification or translation—LLM inference is computationally heavy, memory-intensive, and latency-sensitive. The scale of model parameters, coupled with autoregressive decoding, makes inference efficiency a first-order systems problem.

Modern LLM serving frameworks such as vLLM, and SGLang employ a variety of scheduling and memory-optimization techniques to maximize throughput on shared GPU infrastructure. These systems aim to handle high query loads efficiently while ensuring low latency for user interactions. However, the performance characteristics of these inference systems depend heavily on the structure of the workload—in particular, on how much prefix (context) is shared across different requests.

## 1.1 Motivation

In many real-world scenarios, multiple users query the same underlying document or system prompt. For instance, an Operating Systems teaching assistant bot may maintain a large context consisting of lecture notes and example explanations (typically thousands of tokens), while each student poses short, contextually grounded questions such as "Why does MLFQ need multiple queues?" or "How does the OS handle I/O-bound processes?".

Similarly, enterprise assistants often process hundreds of independent queries over a shared knowledge base or product documentation. In all such cases, the system prompt—representing the long shared context—remains constant, while only the user query varies. These workloads, which we refer to as single-family or shared-prefix inference workloads, are particularly relevant in modern deployments of LLM-based systems.

Despite their prevalence, these workloads expose unique challenges for inference engines: (1) Redundant computation of attention over shared prefixes can waste compute resources (2) Memory footprint grows with increasing batch size and KV cache usage (3) Scheduling efficiency becomes critical when balancing prefill (context processing) and decode (generation) phases

## 1.2 Problem Statement

Our goal in this study is to systematically analyze the performance behavior of LLM inference workloads characterized by shared prefixes—that is, multiple queries with a common large system prompt but distinct suffixes. We seek to answer the following questions:

1. What metrics are relevant to evaluating such inference workloads—both at the GPU and system level?

2. How do modern inference engines (e.g., vLLM, SGLang) handle these workloads internally?

3. How do design decisions such as prefix caching, chunked prefill, and request scheduling affect throughput and latency?

4. What bottlenecks limit scalability as we increase load or number of concurrent families?

## 1.3 Brief Overview of the Work Accomplished

To systematically understand the performance characteristics of LLM inference under shared-prefix workloads, we began by identifying and understanding the metrics and tools most relevant to such analysis. Since LLM inference is ultimately dominated by matrix-matrix multiplications across large tensor dimensions, we first analyzed tiled matrix multiplication kernels. This step allowed us to isolate and interpret GPU-level metrics such as streaming multiprocessor (SM) occupancy, tensor-core utilization and memory throughput. Through this exercise, we also established a baseline familiarity with profiling tools like Nsight Systems and Data Center GPU Management Interface (DCGMI), which were later used to capture traces of inference workloads.

Building upon this foundation, we constructed a controlled inference workload. The setup involved a long shared system prompt—approximately 1500 tokens in length—representing a large contextual document such as a lecture transcript. Over this prompt, we generated multiple short user queries forming a "family" of requests.

Next, we conducted throughput and latency measurements under controlled load conditions. These experiments quantified the achievable requests-per-second throughput and the end-to-end latency for each request. The results served as a baseline for evaluating how the inference engine scales with concurrent load and how effectively it overlaps prefill and decode phases.

To generalize our findings beyond the single-family setting, we extended our experiments to multi-family workloads, where several distinct prefixes coexist. This allowed us to study the interplay between caching, batching, and scheduling in more heterogeneous scenarios that better resemble real-world deployments. Together, these stages form a detailed methodology for evaluating and understanding LLM inference performance across varying degrees of prefix sharing.

## 1.4 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 provides essential background on LLM inference and the architectural foundations necessary to understand system-level optimizations. Chapter 3 presents a detailed overview of the vLLM serving framework, focusing on its memory management techniques and the PagedAttention algorithm that enables efficient key-value (KV) cache utilization. Chapter 4 then discusses SGLang, highlighting its scheduling mechanisms and comparison with vLLM under varying workload structures. Before delving into full inference workloads, Chapter 5 examines matrix multiplication kernels—the core computational unit in transformer-based models—to establish a baseline understanding of GPU-level performance metrics and profiling tools. Chapter 6 describes our initial experiments with smaller system prompt and batches of requests. Subsequently, Chapter 7 and Chapter 8 present our main experiments that evaluate throughput and latency under single-family (shared-prefix) and multi-family workloads, respectively. These analyses highlight how inference engines scale and manage resources across different workload types. Finally, Chapter 9 concludes the report and outlines possible directions for extending this study.

# Chapter 2

# Background

To ground the previous discussion, this section provides the necessary technical background. We begin with the fundamentals of language modeling and transformer architectures, then describe the inference process, optimizations in modern inference engines, and key performance metrics relevant to our study.

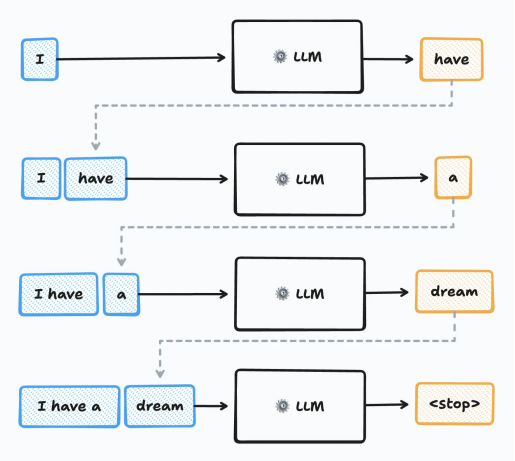



Figure 2.1: LM Model [3]

Language models (LMs) are statistical models that predict the next word in a sequence given the previous words. They basically give us a probability distribution over words based on the context provided by the preceeding words. They enable machines to understand and generate human language. They can be used for a lot of tasks like answering questions, text summarization, translation, code generation and more.

## 2.1 Stages in the Model Creation Pipeline

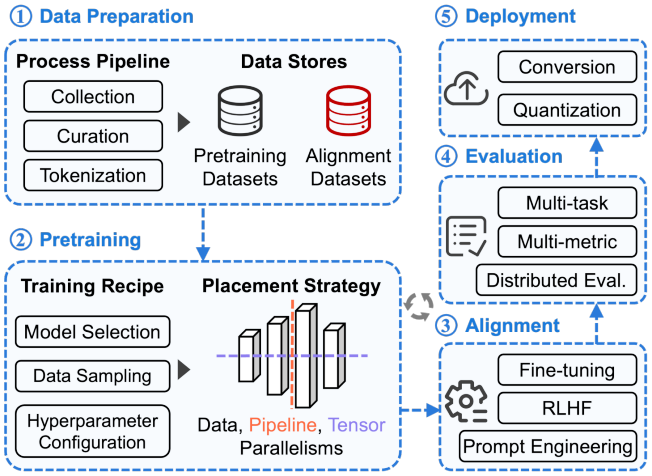



Figure 2.2: Stages in the Model Creation Pipeline [1]

- **Data Preparation:** The initial stage involves gathering and preprocessing the training data, which can be categorized into two parts: (1) pretraining data, consisting of extensive unlabeled corpora obtained from public or private sources and curated through processes like detoxification and deduplication; (2) alignment data, comprising a smaller set of high-quality labeled corpora used to align the model with specific tasks. This data is typically acquired through expensive human annotation or labeling. Besides, all the data must be tokenized to ensure compatibility with the model's input.

- **Pretraining:** It involves self-supervised training on large-scale curated data, demanding a majority of resources within the overall development workflow. Training LLMs efficiently at scale necessitates various system innovations, such as state-sharding optimizers, meticulous model placement using data, pipeline, and tensor parallelisms.

- **Alignment:** This stage aims to adapt LLMs with user intent on a wide range of downstream tasks. Two primary aligning paradigms are commonly used: (1) prompt engineering, specifying prompts (i.e., inputs) without modifying model parameters. For example, in text summarization, appending a prompt "TL; DR" to the input article can improve model performance; (2) fine-tuning, updating model parameters on a task-specific dataset to improve performance in a particular domain.

- **Evaluation:** Given the vast application scenarios of LLM, it may be inaccurate to assess model quality solely based on a single metric like training loss. There are numerous factors to consider, such as accuracy, fairness, and toxicity. Consequently, it is crucial to account for a diverse set of criteria and measure performance across multiple tasks. Furthermore, regular evaluation is essential during the pretraining stage to provide timely feedback on model quality.

- **Deployment:** To meet the strict cost and latency constraints of LLM applications, several advanced techniques have been developed to achieve efficient model serving, including quantization, distillation, CUDA kernel optimization, model parallelism and memory management.

## 2.2 Transformer Architecture

### 2.2.1 Tokenization

Tokenization is the process of converting raw text into a sequence of tokens that the model can process. Instead of working directly with characters or words, modern language models often rely on subword tokenization methods such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece. These approaches break words into smaller, more frequent units, balancing vocabulary size and representation power. For example, the word "unhappiness" may be split into "un", "happiness", while rare words are decomposed into character-level units. Tokenization ensures consistent representation of unseen or rare words and significantly reduces out-of-vocabulary issues.

### 2.2.2 Embedding and Attention

Once tokenized, tokens are mapped into continuous vector representations called embeddings. These embeddings capture semantic and syntactic relationships among words, such that similar words are close in the embedding space. Positional encodings are added to embeddings

to preserve word order information, since attention mechanisms are permutation invariant by default.

Attention is the core mechanism of transformer models. It allows the model to selectively focus on different parts of the input sequence when generating or predicting tokens. Given a query, attention computes weighted similarities with key-value pairs, enabling contextualized representations that consider both local and long-range dependencies.

### 2.2.3 Multi-Head Masked Attention

Masked attention is used in autoregressive language models to prevent the model from "looking ahead" at future tokens during training. This masking enforces causality by ensuring that each token can only attend to itself and preceding tokens. Multi-head attention extends this concept by computing multiple attention functions in parallel. Each head captures different aspects of relationships—such as syntactic dependencies, semantic similarity, or positional cues. The outputs of all heads are concatenated and linearly transformed, providing a richer, more expressive representation of the input.

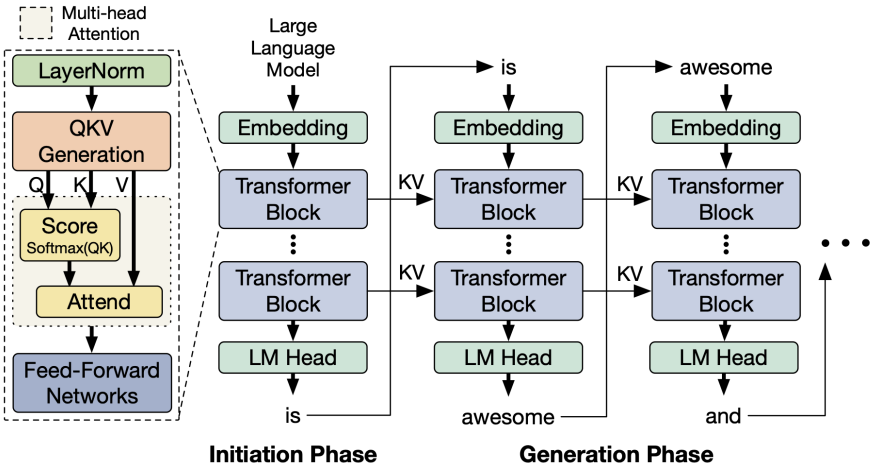



Figure 2.3: Architecture of a Large Language Model [3]

## 2.3 Transformer Based Large Langauge Models

The task of language modeling [11] is to model the probability of a list of tokens $(x_1, \ldots, x_n)$. Since language has a natural sequential ordering, it is common to factorize the joint probability over the whole sequence as the product of conditional probabilities (a.k.a. autoregressive decomposition):

$$P(x) = P(x_1) \cdot P(x_2|x_1) \ldots P(x_n|x_1, \ldots x_{n-1})$$

Transformers have become the de facto standard architecture for modeling the probability above at a large scale. The most important component of a Transformer-based language model is its self-attention layers. For an input hidden state sequence $(x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$, a self-attention layer first applies linear transformations on each position $i$ to get the query, key, and value vectors:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

Then, the self-attention layer computes the attention score $a_{ij}$ by multiplying the query vector at one position with all the key vectors before it and compute the output $o_i$ as the weighted average over the value vectors:

$$a_{ij} = \frac{exp(q_i^T k_j/\sqrt{d})}{\sum_{t=1}^{i} exp(q_i^T k_t/\sqrt{d})}, \quad o_i = \sum_{j=1}^{i} a_{ij} v_j$$

Besides the above computation, all other components in the Transformer model, including the embedding layer, feed-forward layer, layer normalization, residual connection, output logit computation, and the query, key, and value transformation, are all applied independently position-wise in a form of $y_i = f(x_i)$.

- **Query (Q):** A representation of the current token, used to score its relevance against all past tokens. During decoding, we only compute the query for the token currently being processed.
- **Key (K):** Keys act like labels for tokens in the sequence. They are used to match queries and determine which past tokens are relevant.
- **Value (V):** Values are the actual token representations. Once relevance scores are computed (using Q and K), the corresponding values are aggregated to produce the output representation.

### 2.3.1 Autoregressive Generation

A request to an LLM service provides a list of input prompt tokens $(x_1, \ldots, x_n)$,and the LLM service generates a list of output tokens $(x_{n+1}, \ldots, x_{n+T})$. Due to the decomposition, the LLM can only sample and generate new tokens one by one, and the generation process of each new token depends on all the previous tokens in that sequence, specifically their key and value vectors. In this sequential generation process, the key and value vectors of existing tokens are often cached for generating future tokens, known as KV cache. Note that the KV cache of one token depends on all its previous tokens. This means that the KV cache of the same token appearing at different positions in a sequence will be different.

Given a request prompt, the generation computation in the LLM service can be decomposed into two phases:

**The prompt phase** takes the whole user prompt $(x_1, \ldots, x_n)$ as input and computes the probability of the first new token $P(x_{n+1}|x_1, \ldots, x_n)$. During this process, also generates the key vectors $k_1, \ldots, k_n$ and value vectors $v_1, \ldots, v_n$. Since prompt tokens $x_1, \ldots, x_n$ are all known, the computation of the prompt phase can be parallelized using matrix- matrix multiplication operations. Therefore, this phase can efficiently use the parallelism inherent in GPUs. This phase is also known as the **prefill** phase.

**The autoregressive generation phase** generates the remaining new tokens sequentially. At iteration $t$ the model takes one token $x_{n+t}$ as input and computes the probability $P(x_{n+t+1}|x_1, \ldots, x_{n+t})$ with the key vectors $k_1, \ldots, k_{n+t}$ and value vectors $v_1, \ldots, v_{n+t}$. Note that the key and value vectors at positions 1 to $n + t - 1$ are cached at previous iterations, only the new key and value vector $k_{n+t}$ and $v_{n+t}$ are computed at this iteration. This phase completes either when the sequence reaches a maximum length (specified by users or limited by LLMs) or when an end-of-sequence (<eos>) token is emitted. The computation at different iterations cannot be parallelized due to the data dependency and often uses matrix-vector

multiplication, which is less efficient. As a result, this phase severely underutilizes GPU computation and becomes memory-bound, being responsible for most portion of the latency of a single request. This phase is also known as the **decode** phase.

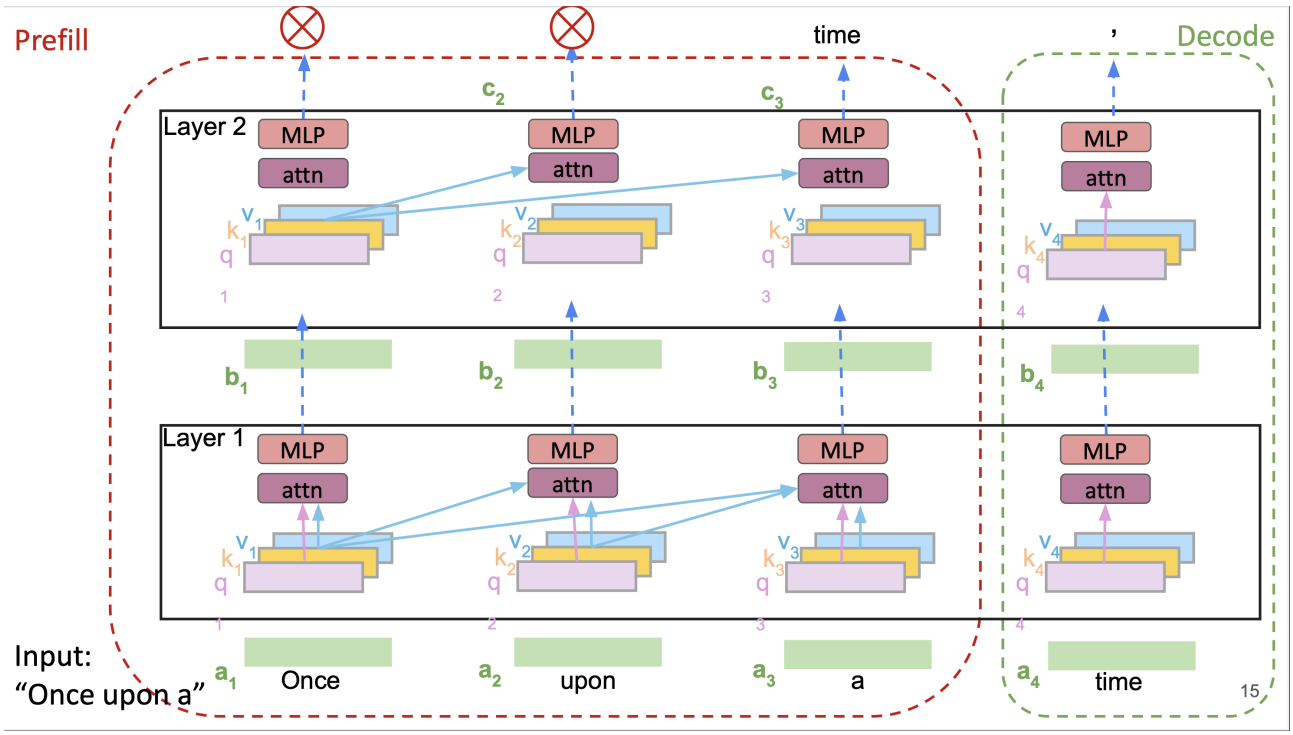



Figure 2.4: Prefill vs Decode [10]

## 2.4 LLM Inferencing

Large Language Model (LLM) inferencing refers to the process of using a trained model to generate predictions (such as text completions, answers, translations, etc.) given some input. Unlike training, where parameters are updated, inferencing only runs the trained model forward to produce outputs.

### 2.4.1 Forward Pass

- The forward pass is the core computation step in inferencing.
- Input tokens (converted to embeddings) are passed layer by layer through the transformer blocks:
  - Embedding → Attention → Feedforward → Normalization (repeated across layers).
- Each forward pass produces *logits*: a vector of unnormalized scores for the next token in the vocabulary.

### 2.4.2 Token Probability Distribution

- The logits are passed through a softmax to get probabilities.
- These probabilities define the model's belief about which token should come next.

- Different decoding strategies may be applied:
  - **Greedy decoding**: pick the token with maximum probability.
  - **Sampling**: sample a token according to the probability distribution.
  - **Top-k / Top-p sampling**: restrict sampling to the most probable tokens.
  - **Beam search**: keep multiple hypotheses alive and expand them.

### 2.4.3 Generation Loop

- Inferencing is auto-regressive:
  1. Generate one token.
  2. Append it to the context.
  3. Feed back into the model.
- Repeat until a stopping criterion (e.g., end-of-sequence token or maximum length) is reached.

### 2.4.4 Modes of Inference

- **Forward only** (logits for a given input): used in classification, scoring, and embeddings.
- **Generate** (auto-regressive token loop): used in chatbots, summarization, and translation.
- **Hybrid tasks**: e.g., retrieval-augmented generation (RAG), where inference integrates external knowledge.

## 2.5 Inference Engines

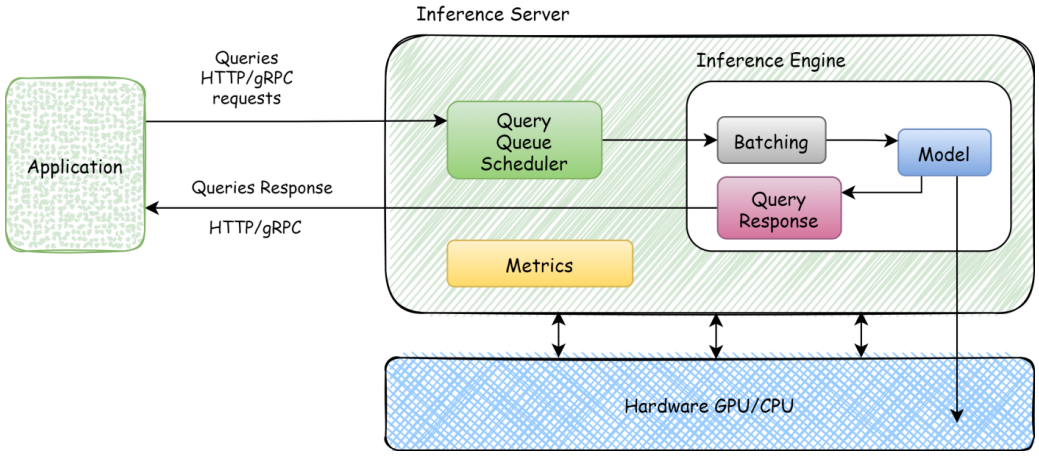



Figure 2.5: Architecture of an Inference Engine [3]

An **inference engine** is the software and systems layer that manages the execution of large language model (LLM) inference on hardware resources. While the model architecture and parameters define the computation, inference engines ensure that requests are executed efficiently, reliably, and at scale.

- **Batching:** Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead. Efficient batching is crucial for achieving high throughput in production environments.
- **Streaming:** Outputs are generated token-by-token and streamed back to the user as they are produced, reducing latency for interactive applications such as chatbots.
- **Fault Management:** Inference engines must be resilient to hardware or software failures. This includes checkpointing, task retries, and robust scheduling.
- **Error Handling:** Engines provide mechanisms for handling issues such as invalid inputs, out-of-memory errors, or timeouts without disrupting other active requests.
- **Parallelism (Multi-GPU):** Large models often cannot fit on a single GPU. Inference engines handle model parallelism and pipeline parallelism to distribute computation across multiple GPUs or nodes, while minimizing communication overhead.

### 2.5.1 Optimizations in Inference Engines

- **Memory Storage Optimizations:** Inference engines carefully manage GPU memory by caching key-value pairs (KV cache), reusing memory buffers, and applying quantization or weight-sharing techniques to reduce footprint.
- **Offloading:** Portions of the model state or intermediate activations can be offloaded to CPU or other storage when not actively needed on the GPU. This allows inference of models larger than GPU memory capacity.

## 2.6 Metrics

The performance of LLM inference is typically measured using two key metrics:

### 2.6.1 Throughput

- Defined as the number of tokens processed or the number of requests completed per second.
- Reflects system efficiency and GPU utilization across multiple requests.

### 2.6.2 Latency

- Measures the responsiveness of the system.
- **Time to first token (TTFT):** Corresponds to the prefill phase, i.e., the delay before the first output token is produced.
- **Time between tokens:** Corresponds to the decode phase, i.e., the per-token delay during sequential generation.

# Chapter 3

# vLLM

vLLM [11] an LLM serving system that achieves (1) near-zero waste in KV cache memory and (2) flexible sharing of KV cache within and across requests to further reduce memory usage. It also proposes PagedAttention, an attention algorithm inspired by the classical virtual memory and paging techniques in operating systems.

Approximately 65% of the memory (for a 13B LLM model) is allocated for the model weights, which remain static during serving. Close to 30% of the memory is used to store the dynamic states of the requests. For Transformers, these states consist of the key and value tensors associated with the attention mechanism. The remaining small percentage of memory is used for other data, including activations - the ephemeral tensors created when performing inference on the LLM. Since the model weights are constant and the activations only occupy a small fraction of the GPU memory, the way the KV cache is managed is critical in determining the maximum batch size.

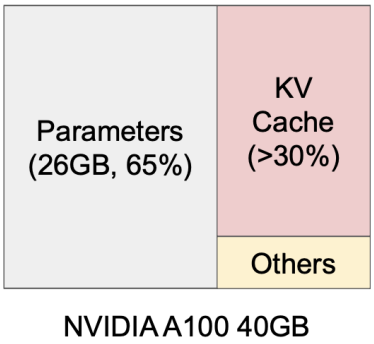



Figure 3.1: Memory layout when serving an LLM with 13B parameters on NVIDIA A100 [11]

Unlike the tensors in the traditional deep learning workloads, the KV cache has unique characteristics: it dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known a priori. These characteristics make the existing systems' approach significantly inefficient in two ways: First, the existing systems suffer from internal and external memory fragmentation. To store the KV cache of a request in contiguous space, they pre-allocate a contiguous chunk of memory with the request's maximum length (e.g., 2048 tokens). This can result in severe internal fragmentation, since the request's actual length can be much shorter than its maximum length. Besides, external memory fragmentation can also be significant, since the preallocated size can be different for each request. Second, the existing systems cannot exploit the opportunities for memory sharing. LLM services often use advanced decoding algorithms, such as parallel sampling and beam search, that generate multiple outputs per request. In these scenarios, the request consists of multiple sequences that can partially share their KV cache. However, memory sharing is not possible in the existing systems because the KV cache of the sequences is stored in separate contiguous spaces.

## 3.1 Memory Challenges

- **Large KV Cache:** The KV Cache size grows quickly with the number of requests. As an example, for the 13B parameter OPT model, the KV cache of a single token demands 800KB of space, calculated as 2 (key and value vectors)× 5120 (hidden state size) × 40 (number of layers) × 2 (bytes per FP16). Since OPT can generate sequences up to 2048 tokens, the memory required to store the KV cache of one request can be as much as 1.6 GB.
- **Complex decoding algorithms:** LLM services offer a range of decoding algorithms for users to select from, each with varying implications for memory management complexity.
- **Scheduling for unknown input & output lengths:** The requests to an LLM service exhibit variability in their input and output lengths. This requires the memory management system to accommodate a wide range of prompt lengths. In addition, as the output length of a request grows at decoding, the memory required for its KV cache also expands and may exhaust available memory for incoming requests or ongoing generation for existing prompts.

## 3.2 vLLM

vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers. The KV cache manager effectively manages the KV cache in a paged fashion, enabled by PagedAttention. Specifically, the KV cache manager manages the physical KV cache memory on the GPU workers through the instructions sent by the centralized scheduler.

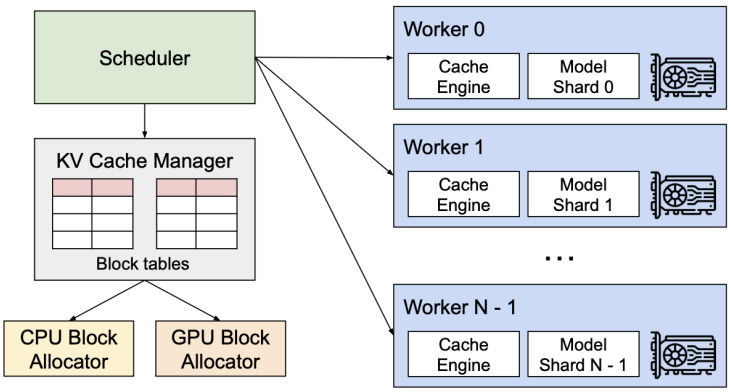



Figure 3.2: vLLM system overview [11]

### 3.2.1 PagedAttention

PagedAttention partitions the KV cache of each sequence into KV blocks. Each block contains the key and value vectors for a fixed number of tokens. During the attention computation, the PagedAttention kernel identifies and fetches different KV blocks separately. In summary, the PagedAttention algorithm allows the KV blocks to be stored in non-contiguous physical memory, which enables more flexible paged memory management in vLLM.

### 3.2.2 KV Cache Manager

A request's KV cache is represented as a series of logical KV blocks, filled from left to right as new tokens and their KV cache are generated. The last KV block's unfilled positions are reserved for future generations. On GPU workers, a block engine allocates a contiguous chunk of GPU DRAM and divides it into physical KV blocks (this is also done on CPU RAM for swapping). The KV block manager also maintains block tables—the mapping between logical

and physical KV blocks of each request. Each block table entry records the corresponding physical blocks of a logical block and the number of filled positions. Separating logical and physical KV blocks allows vLLM to dynamically grow the KV cache memory without reserving it for all positions in advance, which eliminates most memory waste of the existing systems.

### 3.2.3 Parallel Sampling

In parallel sampling, one request includes multiple samples sharing the same input prompt, allowing the KV cache of the prompt to be shared as well. Since both outputs share the same prompt, space is reserved only for one copy of the prompt's state at the prompt phase; the logical blocks for the prompts of both sequences are mapped to the same physical blocks. Since a single physical block can be mapped to multiple logical blocks, a reference count is introduced for each physical block. At the generation phase, the two outputs sample different output tokens and need separate storage for KV cache. vLLM implements a copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences. Unlike parallel decoding, beam search facilities sharing not only the initial prompt blocks but also other blocks across different candidates, and the sharing patterns dynamically change as the decoding process advances.

### 3.2.4 Scheduling and Preemption

Typically, eviction policies use heuristics to predict which block will be accessed furthest in the future and evict that block. Since in our case it's known that all blocks of a sequence are accessed together, we implement an all-or-nothing eviction policy, i.e., either evict all or none of the blocks of a sequence. Furthermore, multiple sequences within one request (e.g., beam candidates in one beam search request) are gang-scheduled as a sequence group. The sequences within one sequence group are always preempted or rescheduled together due to potential memory sharing across those sequences. There are two techniques to recover an evicted block:

#### Swapping

In vLLM's case, evicted blocks are copied to the CPU memory. vLLM includes a CPU block allocator to manage the physical blocks swapped to CPU RAM. When vLLM exhausts free physical blocks for new tokens, it selects a set of sequences to evict and transfer their KV cache to the CPU. Once it preempts a sequence and evicts its blocks, vLLM stops accepting new requests until all preempted sequences are completed. Once a request completes, its blocks are freed from memory, and the blocks of a preempted sequence are brought back in to continue the processing of that sequence. Note that with this design, the number of blocks swapped to the CPU RAM never exceeds the number of total physical blocks in the GPU RAM, so the swap space on the CPU RAM is bounded by the GPU memory allocated for the KV cache.

#### Recomputation

In this case, we simply recompute the KV cache when the preempted sequences are rescheduled. The performances of swapping and recomputation depend on the bandwidth between CPU RAM and GPU memory and the computation power of the GPU.

## 3.3 Automatic Prefix Caching in vLLM

Prefix caching[6] is a key optimization in LLM inference that avoids redundant computation by reusing previously computed `KVCacheBlocks`. When a new request shares a prefix with an earlier one, the system reuses the cached blocks instead of reprocessing the same tokens—resulting in large speedups without changing model outputs. This idea is widely adopted by production systems such as OpenAI and Anthropic, and implemented in open-source frameworks like vLLM and SGLang.

### 3.3.1 Overview

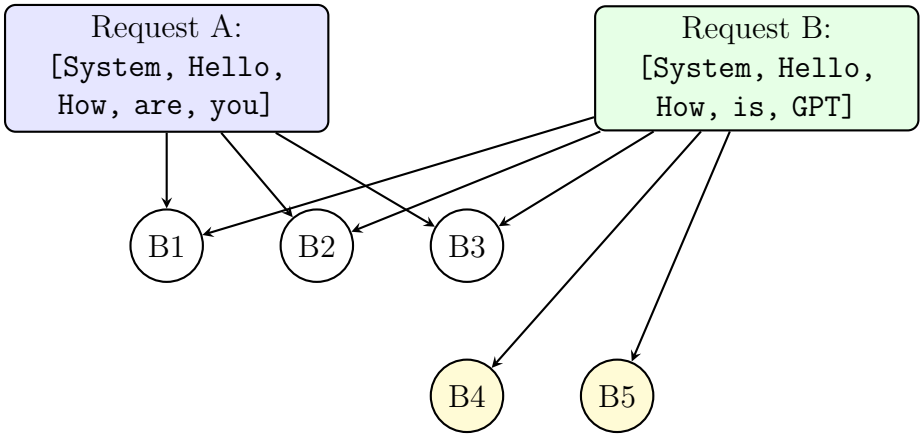



Figure 3.3: Illustration of prefix caching: shared prefixes reuse cached KV blocks.

Each cached block is identified by a unique hash computed from:

- **Parent hash:** hash of the preceding block.
- **Block tokens:** tuple of token IDs in this block (minimizes hash collisions).
- **Extra fields:** LoRA IDs, multi-modality hashes, or tenant-specific salts.

### 3.3.2 System Architecture

The KV cache manager maintains several internal structures:

- **Block Pool:** Pre-allocated list of `KVCacheBlock` instances to avoid dynamic object creation.
- **Free Block Queue:** A doubly linked list that tracks available blocks.
- **Cache Map:** Hash table mapping block hashes to block IDs.
- **Request Map:** Tracks which blocks belong to each active request.

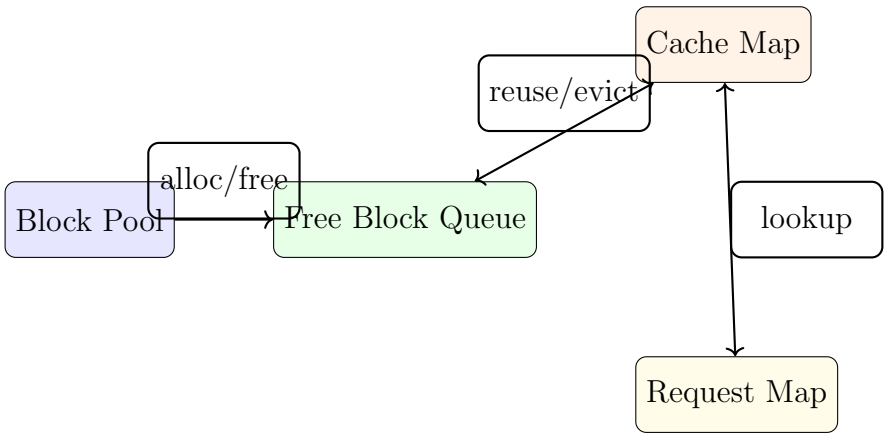



Figure 3.4: Internal organization of the KV cache manager.

### 3.3.3 Core Operations

**1. Block Allocation** When the scheduler processes a new request:

1. It queries precomputed blocks via `get_computed_blocks()` using prefix hashes.
2. For each block:
   - Increment its reference count and remove it from the free queue.
   - Allocate new blocks as needed from the free queue head.
   - If allocation exceeds available capacity, evict the least recently used (LRU) blocks.

**2. Free Operation** After a request completes:

- All blocks with zero references are appended to the tail of the free queue (in reverse order).
- Later blocks, which encode more tokens, are less likely to be reused soon.

**3. Eviction Policy** When the head of the free queue holds a cached block:

1. Remove it from both the queue and cache map.
2. Delete the hash entry to maintain cache consistency.

## 3.4 v1 Scheduler

The v1 scheduler in vLLM follows a unified scheduling policy that treats both prefill and decode phases identically. In each round, it starts with a fixed token budget and first tries to advance all currently running requests in a roughly first-come-first-serve manner. For each request, it determines how many tokens can be processed, and attempts to allocate KV cache blocks through the cache manager. If allocation fails due to limited memory, the scheduler preempts lower-priority requests—freeing their KV blocks—to make room for higher-priority ones. Once all running requests are handled (or the token budget is exhausted), the scheduler admits waiting requests if capacity remains, moving them into the running queue and allocating KV cache space. This process continues iteratively each round, ensuring that GPU resources

are efficiently utilized while managing KV cache pressure through controlled preemption and reuse. Here is a rough outline of the scheduling algorithm:

**Algorithm 1: Scheduler::schedule()**

**Require:** current state: `running`, `waiting`, `kv_cache_manager`, config/state variables

Initialize lists:
`scheduled_new_reqs` $\leftarrow []$
`scheduled_resumed_reqs` $\leftarrow []$
`scheduled_running_reqs` $\leftarrow []$
`preempted_reqs` $\leftarrow []$
Initialize maps:
`req_to_new_blocks` $\leftarrow \{\}$, `num_scheduled_tokens` $\leftarrow \{\}$

`token_budget` $\leftarrow$ `self.max_num_scheduled_tokens` ▷ Max number of tokens we can schedule in this round

▷ Schedule RUNNING requests (FCFS with preemption on allocation failure)

`req_index` $\leftarrow 0$
**while** `req_index` $<$ len(`running`) $\wedge$ `token_budget` $> 0$ **do**
  `request` $\leftarrow$ `running[req_index]`
  `num_new_tokens` $\leftarrow$ `request.num_tokens_with_spec+request.num_output_placeholders-request.num_computed_tokens`
  `num_new_tokens` $\leftarrow \min($`num_new_tokens`, `token_budget`$)$
  **if** `num_new_tokens` $= 0$ **then**
    `req_index` $\leftarrow$ `req_index` $+ 1$ ▷ try other requests (not strict FCFS)
    **continue**
  **end if**
  ▷ Attempt to allocate KV cache blocks for this chunk (may preempt others)
  **repeat**
    `new_blocks` $\leftarrow$ `kv_cache_manager.allocate_slots(request, num_new_tokens, num_lookahead_tokens)`
    **if** `new_blocks` $=$ `None` **then** ▷ Allocation failed $\Rightarrow$ preempt a lower-priority request if possible
      `preempted_req` $\leftarrow$ `kv_cache_manager.select_preemptible_req()` ▷ (selection policy)
      **if** `preempted_req` $\neq$ `None` **then**
        `kv_cache_manager.free(preempted_req)`
        `waiting.prepend_request(preempted_req)`
        **if** `preempted_req` $=$ `request` **then**
          `can_schedule` $\leftarrow$ `false`
          **break**
        **end if**
      **else**
        `can_schedule` $\leftarrow$ `false`
      **end if**
    **else**
      `can_schedule` $\leftarrow$ `true`
    **end if**
  **until** `can_schedule` $=$ `true`
  **if** `can_schedule` $=$ `true` **then**
    `scheduled_running_reqs.append(request)`

```
        req_to_new_blocks[request.id] ← new_blocks
        num_scheduled_tokens[request.id] ← num_new_tokens
        token_budget ← token_budget − num_new_tokens
        req_index ← req_index + 1
    else
        break                                  ▷ Could not schedule this running request now
    end if
end while
                                  ▷ Schedule WAITING requests (skip only if something was preempted)
skipped_waiting_requests ← create_request_queue(policy)
if preempted_reqs = [] then
    while waiting not empty ∧ token_budget > 0 do
        if len(running) = max_num_running_reqs then
            break
        end if
        request ← waiting.peek_request()
                                  ▷ Decide how many new tokens to schedule for this waiting request
        num_new_tokens ← request.num_tokens − request.num_computed_tokens
        if ¬scheduler_config.chunked_prefill_enabled∧num_new_tokens > token_budget
then
            waiting.pop_request()
            skipped_waiting_requests.prepend_request(request)
            continue
        end if
        num_new_tokens ← min(num_new_tokens, token_budget)
          ▷ Allocate KV blocks for waiting request (includes already-cached local computed
blocks)
                                  ▷ Admit the request: pop from waiting and append to running
        request ← waiting.pop_request()
        append request to running
        req_to_new_blocks[request.id] ← kv_cache_manager.get_blocks(request.id)
        num_scheduled_tokens[request.id] ← num_new_tokens
        token_budget ← token_budget − num_new_tokens
        set request.status ← RUNNING
        set request.num_computed_tokens ← request.num_computed_tokens
        if request.num_cached_tokens < 0 then
            set request.num_cached_tokens ← request.num_computed_tokens
        end if
    end while
end if
                                  ▷ Restore skipped waiting requests to the head of waiting queue
                                                  ▷ Sanity checks and prepare output
return scheduler_output
```

# Chapter 4

# SGLang

SGLANG [13] is a system for efficient execution of complex language model programs. SGLang consists of a frontend language and a runtime. The frontend simplifies programming with primitives for generation and parallelism control. The runtime accelerates execution with novel optimizations like RadixAttention for KV cache reuse and compressed finite state machines for faster structured output decoding.

## 4.1 LM Programs

The emergence of few-shot learning, self-consistency, skeleton-of-thought, tree-of-thought and various other patterns signifies a shift in our interaction with LLMs, moving from simple chatting to a more sophisticated form of programmatic usage of LLMs, which means using a program to schedule and control the generation processes of LLMs. These programs are referred to as LM (Language Model) programs. There are two common properties of LM programs:

- LM programs typically contain multiple LLM calls interspersed with control flow. This is needed to complete complex tasks and improve overall quality.
- LM programs receive structured inputs and produce structured outputs. This is needed to enable the composition of LM programs and to integrate LM programs into existing software systems.

## 4.2 Programming Model

**Language primitives:** SGLang provides primitives for controlling prompt state, generation, and parallelism. Here are the primitives: “gen” calls a model to generate and stores the results in a variable with the name specified in its first argument. It supports a “regex” argument to constrain the output to follow a grammar defined by a regular expression (e.g., a JSON schema). “select” calls a model to choose the highest probability option from a list. The operator “+=” or “extend” appends a string to the prompt. The operator “[variable_name]” fetches the results of a generation. “fork” creates parallel forks of the prompt state. “join” rejoins the prompt state. “image” and “video” take in image and video inputs.

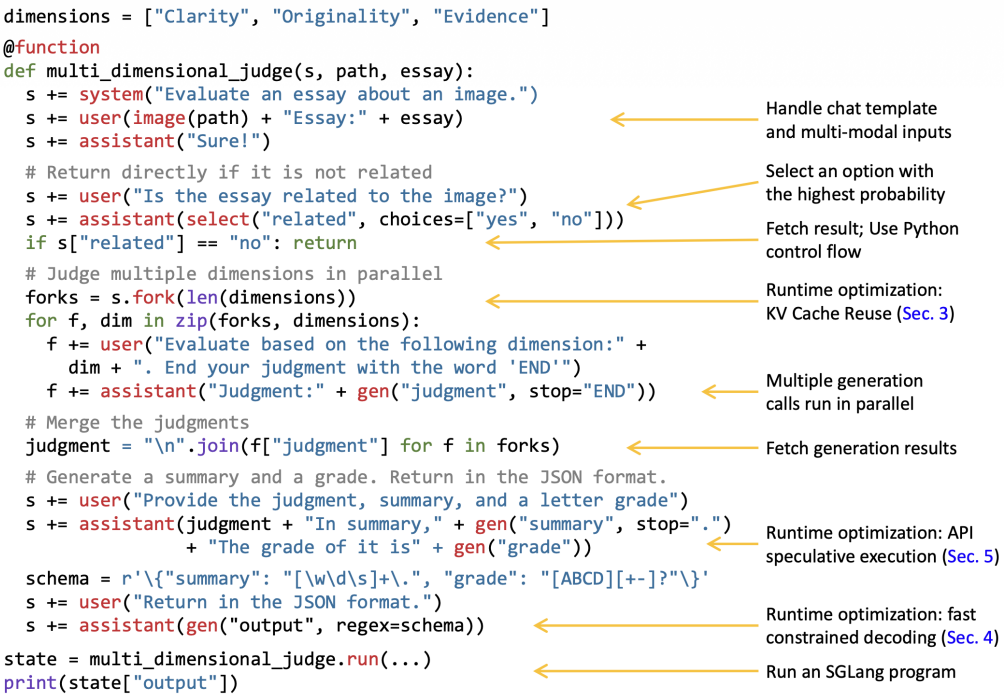



Figure 4.1: SGLANG Frontend Language Model [13]

## 4.3 RadixAttention

A radix tree is a data structure that serves as a space-efficient alternative to a classical trie (prefix tree). Unlike typical trees, the edges of a radix tree can be labeled not just with single elements but also with sequences of elements of varying lengths, significantly enhancing efficiency. In SGLANG, a radix tree is utilized to manage a mapping between sequences of tokens, and their corresponding KV cache tensors. These KV cache tensors are stored in a non-contiguous, paged layout, where the size of each page is equivalent to one token. Because GPU memory is quickly filled by the KV cahce, a simple LRU eviction policy is introduced that evicts the least recently used leaf first. By evicting leaves first, we enable the re-use of their common ancestors until those ancestors become leaves and are also evicted.

In the continuous batching setting, we cannot evict nodes used by the currently running batch. Each node maintains a reference counter indicating how many running requests are using it. A node is evictable if its reference counter is zero.

The cache hit rate is defined as $\frac{\texttt{number of cached prompt tokens}}{\texttt{number of prompt tokens}}$. If the request scheduler frequently switches between different, unrelated requests, it can lead to cache thrashing and a low hit rate.

In the batch-processing setting the requests are sorted by matched prefix length and prioritize requests with longer matched prefixes instead of using a first-come, first-served schedule.

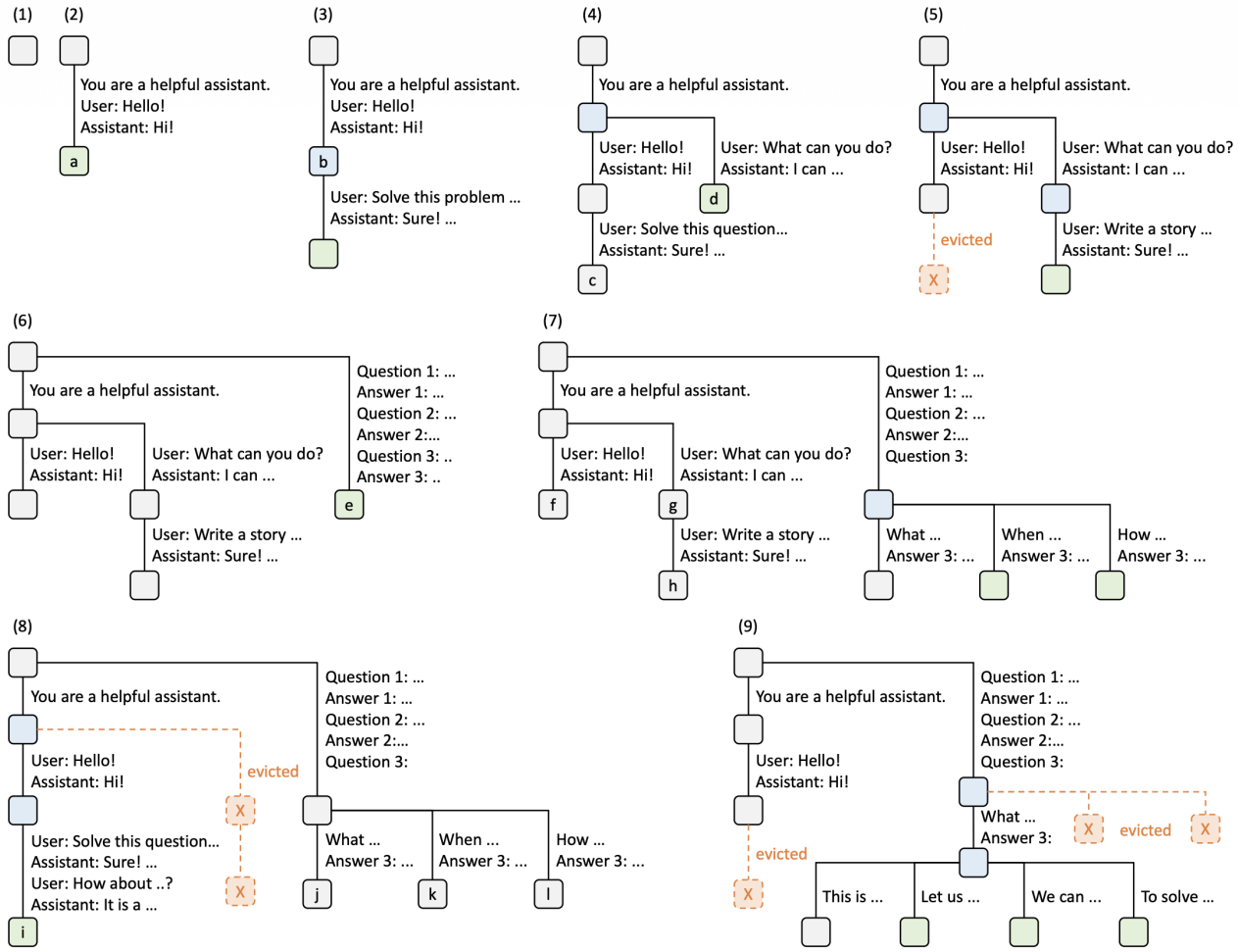



Figure 4.2: SGLANG Radix Tree Example [13]

## 4.4 Compressed Finite State Machine

In LM programs, users often want to constrain the model's output to follow specific formats, such as JSON schemas. Existing systems support this by converting a regular expression into a finite state machine (FSM). During decoding, they maintain the current FSM state, retrieve allowed tokens from the next states, and set the probability of invalid tokens to zero, decoding token by token. They can only decode one token at a time because the lack of integration between the FSM and the model runner in existing systems prevents multi-token processing, resulting in slow decoding. SGLang overcomes this limitation by creating a fast constrained decoding runtime with a compressed FSM. This runtime analyzes the FSM and compresses adjacent singular-transition edges in the FSM into single edges.

## 4.5 FastTree: Optimizing Radix Tree based KV Cache Computation

Existing systems still use conventional attention kernels that concatenate the KV cache per query and process each query independently. This leads to redundant memory loads of shared contexts from slow global memory and poor tensor core utilization due to mismatched tile dimensions.

FastTree [12] introduces GPU kernels specifically designed to process queries that share common prefixes within a radix tree structure. By adapting the computation to the tree topology at runtime, FastTree eliminates redundant memory operations and improves throughput.

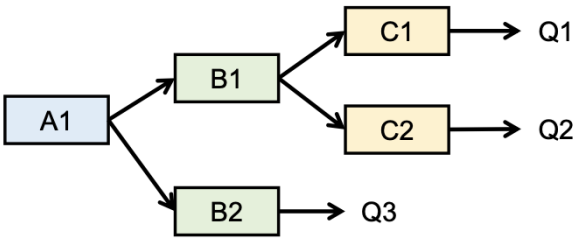



Figure 4.3: KV cache sharing through Radix Tree [12]

**Tree-structure Adaptive Runtime Optimization:** Different ways of grouping queries—based on how prefixes are merged—can lead to drastically different computation patterns and memory-sharing behavior. Even with identical tree topologies, certain grouping plans might waste computation due to tile padding or cause additional global memory transactions when intermediate results must be written out. FastTree formulates this as an optimization problem: given a KV cache tree $T = (V, E)$, it searches for an edge assignment function $f : E \to \{0, 1\}$ that minimizes the end-to-end latency of the GPU attention kernel, balancing the trade-off between computation reuse and intermediate data cost.

Padding overhead arises when the number of queries or context tokens fails to meet the tensor core tile size, wasting both shared memory and FLOPs. This is modeled through cost functions $C_{P,q}$ and $C_{P,c}$, which estimate padding-induced inefficiency at the query and context dimensions respectively. Similarly, splitting the KV matrices introduces intermediate reduction overheads, modeled as SplitKV_Cost, capturing both padding and extra memory I/O cost. FastTree employs a greedy runtime algorithm (Fig. 4.4) that evaluates these costs per grouping plan and dynamically selects the most efficient configuration.

**Efficient Attention Kernel Design:** After determining the optimal grouping plan, FastTree performs the attention computation tile-by-tile within each group. It parallelizes the tiles of the query (Q) matrix across thread blocks, while iterating over key (K) and value (V) tiles sequentially within each block due to inter-tile dependencies. By caching QKV tiles in shared memory and applying an online softmax, FastTree minimizes global memory transactions. Furthermore, aggregating multiple queries enables the transformation of GEMV into GEMM operations, thereby unlocking high tensor core utilization on modern GPUs.

Overall, FastTree demonstrates that by coupling radix tree–based KV cache organization with adaptive runtime planning and GPU-aware attention kernels, significant reductions in redundant computation and memory access can be achieved, leading to markedly higher throughput and lower latency for shared-context inference workloads.

**Algorithm 1** Greedy Binary Edge Assignment

**Input:** Radix tree $T = (V, E)$

Initialize $A$ as the number of all associated queries of each node in $V$
Initialize $L$ as the context length of each node
**for** node $v$ in BFS($V$) **do**
  $nQ_{curr} = A[v]$ ▷ current #aggregated queries
  $len_v = L[v]$ ▷ accumulated context length
  **for** $l$ in leaves($v$) **do**
    $nQ_l = A[l]$, $len_l = L[l]$
    $C_0 = \text{SplitKVCost}(nQ_{curr}, nQ_l, len_l, len_v)$
    $C_1 = \text{SplitQCost}(nQ_{curr}, nQ_l, len_l, len_v)$
    **if** $C_0 \geq C_1$ **then**
      Assign 1 to edge $v \to l$
      $nQ_{curr} = nQ_{curr} - nQ_l$
      $L[l] = len_l + len_v$
    **else**
      Assign 0 to edge $v \to l$
    **end if**
  **end for**
**end for**

Figure 4.4: Greedy runtime optimization algorithm [12]

# Chapter 5

# Matrix Multiplication Analysis

## 5.1 Motivation

Large Language Model (LLM) inference workloads are dominated by matrix multiplication operations that form the core of attention and feed-forward layers. However, the overall behavior of such workloads on GPUs is influenced by an interplay of compute, memory, and communication characteristics. Before analyzing end-to-end inference pipelines, it is important to develop a clear understanding of how fundamental GPU metrics behave in a controlled and interpretable setting.

To this end, we begin with the tiled matrix multiplication workload—a simplified kernel that captures the essential compute and memory access patterns seen in LLM inference. The goal here is not to optimize matrix multiplication itself, but rather to use it as a microbenchmark to study key GPU metrics, identify relevant performance indicators, and familiarize ourselves with profiling tools that will later be applied to real inference runs.

We employ two primary NVIDIA tools for this study: **Nsight Systems (NSYS)** and the **Data Center GPU Management Interface (DCGMI)**. NSYS provides fine-grained timeline traces, kernel-level statistics, and PCIe throughput data, while DCGMI offers continuous runtime counters such as SM activity, tensor utilization, and memory bandwidth usage. Together, these tools allow us to correlate different phases of GPU execution—data transfers, kernel computation, and synchronization—with corresponding hardware metrics.

By first examining matrix multiplication in isolation, we build intuition about how metrics such as SM occupancy, memory bandwidth utilization, and tensor activity reflect GPU efficiency. This understanding serves as a foundation for interpreting the more complex, multi-kernel behavior of LLM inference workloads analyzed later.

## 5.2 Algorithm

The CUDA kernel `matmul_tiled` computes the product of two tiles and accumulates the result in the output tile. Each thread block computes a $B_s \times B_s$ submatrix of the output tile.

**Algorithm 1** Tiled Matrix Multiplication using CUDA with Host-Level Tiling

**Require:** Matrices $A, B \in \mathbb{R}^{N \times N}$, block size $B_s$, tile size $T_s$
**Ensure:** $C = A \times B$

Allocate host matrices $h_A, h_B, h_C$
▷ Process in tiles at host level
**for** $i = 0$ to $N - 1$ **step** $T_s$ **do**
  **for** $j = 0$ to $N - 1$ **step** $T_s$ **do**
    Allocate device submatrix $d_C$ of size $T_s \times T_s$
    Initialize $d_C \leftarrow 0$
    **for** $k = 0$ to $N - 1$ **step** $T_s$ **do**
      Allocate $d_A, d_B$ on device $(T_s \times T_s)$
      Copy $T_s \times T_s$ submatrices:
        $d_A \leftarrow h_A[i : i + T_s,\ k : k + T_s]$
        $d_B \leftarrow h_B[k : k + T_s,\ j : j + T_s]$
      Launch CUDA kernel:
        matmul_tiled $<<<$ $\lceil T_s/B_s \rceil$, $\lceil T_s/B_s \rceil$ $>>>$ $(d_A, d_B, d_C, T_s)$
      Synchronize device
      Free $d_A, d_B$
    **end for**
    Copy $d_C$ back to host:
      $h_C[i : i + T_s,\ j : j + T_s] \leftarrow d_C$
    Free $d_C$
  **end for**
**end for**
**return** $h_C$

## 5.3 Nsight Systems GPU Metrics

Below is an example of NSYS GPU metrics captured during a matrix multiplication run.

| Metric | Description |
|---|---|
| GPC Clock Frequency | GPU core clock (Graphics Processing Cluster). Determines SM execution speed. |
| SYS Clock Frequency | System clock domain of the GPU, affecting memory controllers, PCIe, and copy engines. |
| GR Active | Fraction of time the graphics engine is active (rendering or compute dispatch). |
| Sync Compute In Flight | Number of synchronous compute kernels currently executing (blocking). |
| Async Compute In Flight | Number of asynchronous compute kernels currently executing in parallel (non-blocking). |
| SM Active | Fraction of time at least one warp is active on any Streaming Multiprocessor (SM). |
| SM Issue Instructions | Fraction of cycles where SMs issued at least one instruction. Indicates pipeline utilization. |
| Tensor Active Instructions | Fraction of executed instructions that used Tensor Cores (e.g., matrix multiply). |
| SM Warp Occupancy | Ratio of active warps to the maximum possible per SM. Measures parallel utilization. |
| DRAM Bandwidth | Achieved global memory bandwidth (GB/s). Indicates memory throughput and potential bottlenecks. |
| PCIe RX/TX Throughput | Host-to-Device (RX) and Device-to-Host (TX) data transfer rates over PCIe. |
| PCIe Read/Write BAR1 Requests | CPU requests to GPU memory aperture (BAR1). Reads = CPU fetching from GPU, Writes = CPU writing to GPU. |

Table 5.1: Nsight Systems GPU Metrics

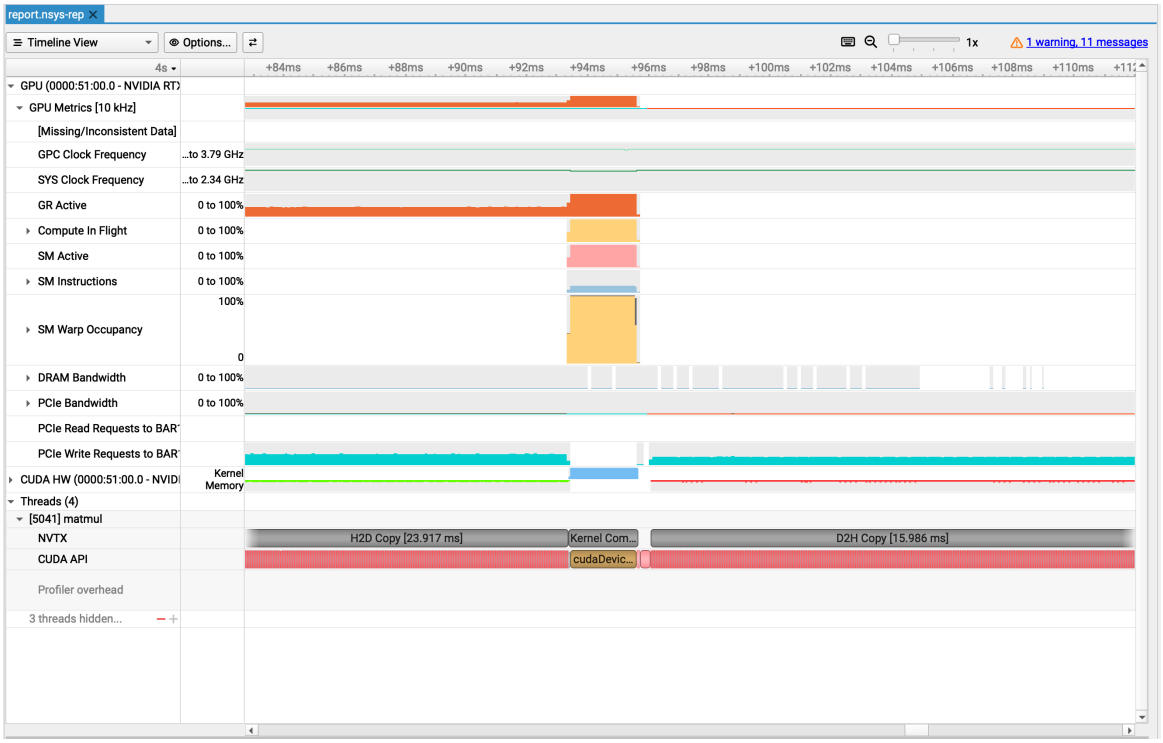



Figure 5.1: NSYS GPU Metrics for a matrix multiplication run

- As shown below by the NVTX ranges, the matrix multiplication involves various phases: host to device copy, kernel computation, and device to host copy.
- The compute metrics - compute in flight, SM Active, SM Occupancy reach high values during the kernel execution phase, indicating good GPU utilization.
- The PCIE bandwidth metrics spike during the copy phases, showing data transfer activity.

## 5.4 Analysis of GPU Utilization using DCGMI

DCGMI - NVIDIA's Data Center GPU Management Interface - also provides insights into GPU performance metrics.

A few important metrics to consider are [4]:

| Metric | Description |
|---|---|
| Graphic Engine Activity | The fraction of time any portion of the graphics or compute engines were active. |
| SM Activity | The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors. Note that "active" does not necessarily mean a warp is actively computing. For instance, warps waiting on memory requests are considered active. |
| SM Occupancy | The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor. The value represents an average over a time interval and is not an instantaneous value. Higher occupancy does not necessarily indicate better GPU usage. |
| Tensor Activity | The fraction of cycles the tensor (HMMA / IMMA) pipe was active. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of the Tensor Cores. |
| Memory BW Utilization | The fraction of cycles where data was sent to or received from device memory. The value represents an average over a time interval and is not an instantaneous value. |
| NVLink Bandwidth | The rate of data transmitted / received over NVLink, not including protocol headers, in bytes per second. |
| PCIe Bandwidth | The rate of data transmitted / received over the PCIe bus, including both protocol headers and data payloads, in bytes per second. |

Table 5.2: DCGMI GPU Metrics

In the figures below, we analyze the GPU utilization metrics (primarily focussed on compute) across different matrix sizes for our tiled matrix multiplication implementation (tile size set equal to the matrix size).

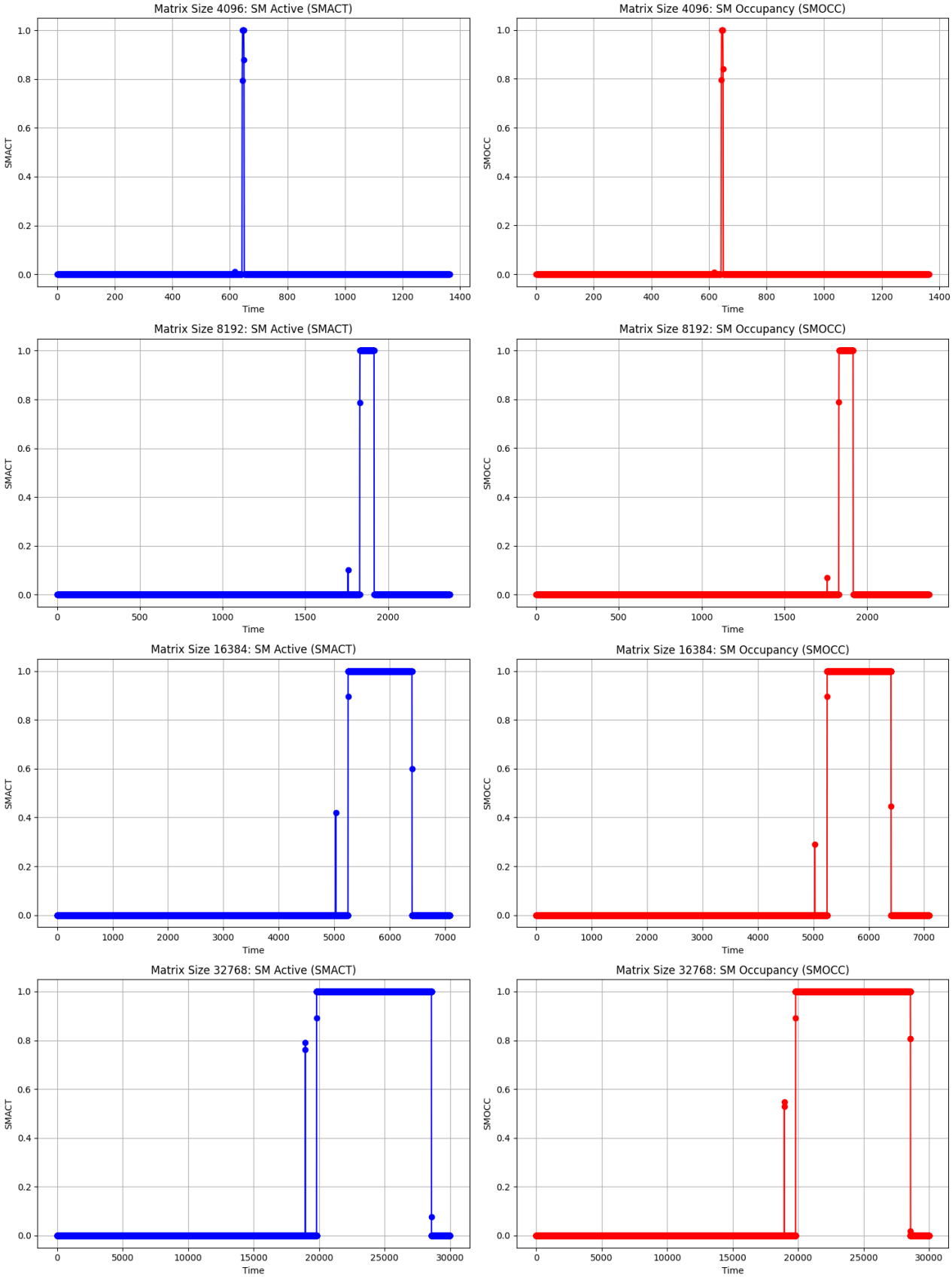



Figure 5.2: SM Active and SM Occupancy across different matrix sizes

# Chapter 6

# Initial Experiments

## 6.1 System Prompt in Cache Experiment

The basic application which we want to cater is inferencing in cases where the system prompt is common across multiple queries. And therefore, we want to utilize caching by already filling the KV-Cache with the system prompt, and then putting user queries on the top of it.

### 6.1.1 Example 1

I picked the 200-word summary of the demand paging lecture as the system prompt to be provided to the LLM. The model which we used was `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1.1 billion parameters). The system prompt provided was:

> The lecture discusses demand paging, a key concept in virtual memory management. Instead of loading all pages of a process into memory, only the required pages are brought in, while the rest may stay in swap space on disk. This allows processes to use address spaces larger than physical memory and enables efficient memory utilization.
> Each process has a page table, with entries containing a valid bit (indicating whether the page is part of the process) and a present bit (whether it is currently in memory). If valid=0, the process makes an illegal access. If valid=1 but present=0, a page fault occurs. The OS then allocates a memory frame, possibly evicts a victim page using a page replacement policy, updates the page table, and resumes execution.
> Victim pages may be file-backed or anonymous, and may be dirty (modified) or clean. Dirty pages require writing back to disk before eviction. Policies like FIFO and LRU determine which pages to evict; LRU is effective due to locality, while FIFO can suffer from Belady's anomaly.
> Excessive page faults can cause thrashing, where the system spends more time swapping pages than doing useful work. To prevent this, the OS ensures that each process's working set fits in memory.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.339 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

> **Query 1:** What is the difference between a valid bit and a present bit?

> **Response 1:** The difference between a valid bit and a present bit is that a valid bit indicates that a page is part of a process, while a present bit indicates that a page is currently in memory.

**Query 2:** Why does thrashing occur in demand paging systems?

**Response 2:** Thrashing occurs in demand paging systems because excessive page faults can cause the system to spend more time swapping pages than doing useful work. This is because demand paging systems allocate memory frames to processes based on their page table entries, which are either valid or present. If a process has a page fault, it must be allocated a memory

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.1 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.94 seconds.

### 6.1.2 Example 2

Let us take another example, this time with a summary of the semaphores lecture. The system prompt provided was:

The lecture introduces semaphores, a synchronization primitive similar to condition variables, used to coordinate threads. A semaphore maintains a counter, initialized to some value. The down (wait) operation decrements the counter and blocks the thread if it becomes negative, while the up (post) operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations.
Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They are also useful for synchronization between threads, e.g., enforcing execution order (T1 before T2) or ensuring two threads finish one step before starting another. Incorrect usage can cause deadlocks, such as when both threads wait on each other.
The producer-consumer problem is revisited using semaphores. Two semaphores track empty and filled buffer slots, while a binary semaphore (mutex) ensures mutual exclusion on buffer access. Misuse, like holding locks during blocking down calls, can also lead to deadlocks.
Guidelines emphasize careful initialization, ensuring every down has a corresponding up, and using separate semaphores for signaling and locking. The lecture also covers batched processing with semaphores, showing how request threads and a batch processor can coordinate efficiently.
Semaphores thus provide flexible yet tricky synchronization, requiring careful design to avoid deadlocks.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.347 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

**Query 1:** How does the down operation on a semaphore synchronize threads during execution?

**Response 1:** A semaphore is a synchronization primitive that maintains a counter, initialized to some value. The down operation decrements the counter and blocks the thread if it becomes negative. The up operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations. Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They

**Query 2:** Why is an extra binary semaphore (mutex) needed in producer-consumer implementation?

**Response 2:** An extra binary semaphore (mutex) is needed in producer-consumer implementation because the producer and consumer threads can access the same buffer at the same time. This can lead to deadlocks if the buffer is not properly protected.

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.774 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.957 seconds.

### 6.1.3 Observations

- As we can see, there is roughly an improvement of 0.2-0.7 seconds for an experiment of 2 queries, which is also intutively clear, because we are just passing over the system prompt once.
- As we increase the size of the system prompt, and the number of tokens we generate, the difference becomes even wider.
- The time taken to feed through the system prompt, and cloning the KV cache values, can be ignored safely because we can do this before we get the actual user queries.
- The first query of the second example didn't get a superior response, however this is independent from our experiment settings, and depends more on the type of model we use, for example increasing the number of parameters (using a larger model) would definitely help.

### 6.1.4 Analysis over different steps and using different metrics

I now use a profiler to actually figure out at what all places, the most amount of time is being spent:

Table 6.1: Profiler Output

| Name | Self CPU % | Self CPU | CPU total % | CPU total | CPU time avg | Self CUDA | Self CUDA % | CUDA total | CUDA time avg | # of Calls |
|---|---|---|---|---|---|---|---|---|---|---|
| aten::matmul | 2.67% | 56.256ms | 30.07% | 632.976ms | 40.575us | 0.000us | 0.00% | 589.952ms | 37.817us | 15600 |
| aten::linear | 1.16% | 24.342ms | 30.78% | 647.989ms | 41.806us | 0.000us | 0.00% | 589.802ms | 38.052us | 15500 |
| aten::mm | 14.46% | 304.341ms | 22.68% | 477.495ms | 30.806us | 579.871ms | 69.41% | 589.802ms | 38.052us | 15500 |
| cutlass kernel (relu bf16) | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 234.992ms | 28.13% | 234.992ms | 65.385us | 3594 |
| ampere_bf16_128x64_sliced1x2 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 124.274ms | 14.87% | 124.274ms | 66.457us | 1870 |
| ampere_bf16_128x128_stages_32x1 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 94.240ms | 11.28% | 94.240ms | 44.621us | 2112 |
| aten::mul | 6.55% | 137.891ms | 10.29% | 216.715ms | 10.728us | 61.543ms | 7.37% | 61.560ms | 3.048us | 20200 |
| aten::copy_ | 3.97% | 83.637ms | 8.32% | 175.218ms | 9.679us | 50.846ms | 6.09% | 50.857ms | 2.809us | 18103 |
| ampere_bf16_128x64_stages_64x3 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 44.572ms | 5.34% | 44.572ms | 19.481us | 2288 |
| aten::scaled_dot_product_attention | 0.84% | 17.683ms | 5.51% | 116.049ms | 52.749us | 0.000us | 0.00% | 44.127ms | 20.058us | 2200 |
| aten::_scaled_dot_product_flash_attention | 0.63% | 13.224ms | 4.67% | 98.366ms | 44.712us | 0.000us | 0.00% | 44.127ms | 20.058us | 2200 |
| aten::_flash_attention_forward | 1.00% | 21.077ms | 3.49% | 73.410ms | 33.368us | 44.073ms | 5.28% | 44.127ms | 20.058us | 2200 |
| flash_fwd_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 43.833ms | 5.25% | 43.833ms | 20.125us | 2178 |
| elementwise_mul_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 37.673ms | 4.51% | 37.673ms | 2.833us | 13300 |
| ampere_bf16_64x64_sliced1x4 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 34.059ms | 4.08% | 34.059ms | 28.669us | 1188 |
| aten::clone | 0.87% | 18.407ms | 6.48% | 136.324ms | 15.491us | 0.000us | 0.00% | 26.201ms | 2.977us | 8800 |
| elementwise_copy_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 26.198ms | 3.14% | 26.198ms | 2.977us | 8800 |
| aten::add | 4.45% | 93.743ms | 7.17% | 150.912ms | 11.347us | 24.648ms | 2.95% | 24.657ms | 1.854us | 13300 |
| aten::to | 0.93% | 19.568ms | 8.33% | 175.368ms | 12.345us | 0.000us | 0.00% | 24.656ms | 1.736us | 14205 |
| aten::_to_copy | 1.29% | 27.125ms | 7.40% | 155.800ms | 16.747us | 0.000us | 0.00% | 24.656ms | 2.650us | 9303 |

Now we go over each of the rows, and further analyze the values:

1. `atten::matmul` = Pytorch's high level matrix multiplication. Runs on CPU side, but calls matrix multiply functions (`atten::mm`) on CUDA.

2. `atten::linear` = The linear layer, has large CUDA compute
3. `cutlass kernel (relu bf16)` = Stands for general matrix multiplication, entirely on CUDA
4. `ampere_...` = Specialized Ampere architecture GEMM kernel in BF16 precision for different tile sizes
5. `atten::mul` = Element-wise matrix multiplication, very high call count
6. `aten::copy_` = Copying tensors, very high call count
7. `aten::_flash_attention_forward` = The forward pass of flash attention

### 6.1.5 Profiling across pipeline stages

The profiling breakdown indicates that the *system forward pass* (KV cache population) dominates the total time cost, accounting for nearly half of the total latency. Tokenization and decoding operations are negligible in comparison. The initial token generation is slower (0.0390 s) due to context initialization, while subsequent tokens average only 0.0148 s, reflecting stable autoregressive inference. Overall, computation-heavy attention and matrix operations during KV cache fill remain the primary bottlenecks.

| **Stage** | **Time (s)** |
|---|---|
| Tokenize system prompt | 0.0052 |
| System forward (KV cache fill) | 0.3342 |
| Tokenize query | 0.0003 |
| Per token (mean / first) | 0.0148 / 0.0390 |
| Generation total | 0.7517 |
| Decode tokens | 0.0002 |

Table 6.2: Profiling results for various pipeline stages

## 6.2 Batch of Requests

We measure the total time taken to process a batch of requests by varying the batch size. Batching significantly improves throughput for the `Transformers` baseline, with performance saturating beyond a batch size of three. However, systems optimized for dynamic batching and KV cache reuse — such as `vLLM`, `SGLang`, and `NEO` — outperform the standard transformer pipeline by over an order of magnitude. Notably, `SGLang (run_batch)` achieves the lowest latency (0.40 s), suggesting the heavy benefit which we get from prefix caching. The results emphasize the critical role of request-level parallelism and KV cache sharing in scaling inference workloads efficiently.

| **Technique Used** | **Time (s)** |
|---|---|
| Transformers (batch size = 1) | 30.21 |
| Transformers (batch size = 2) | 19.51 |
| Transformers (batch size = 3) | 18.64 |
| vLLM (no system prompt pre-filling) | 0.45 |
| SGLang (fork) | 1.03 |
| SGLang (run_batch) | 0.40 |
| NEO | 0.55 |

Table 6.3: Time taken to process a batch of requests using different techniques

# Chapter 7

# Experiments - Single Family of Requests

## 7.1 Throughput and Latency Measurements across different techniques

To systematically understand the performance characteristics of LLM inference engines under such single-family workloads, we begin by quantifying throughput, latency, and GPU resource utilization across representative inference backends.

Here is the experiment setup:

- Model: `TinyLlama/TinyLlama-1.1B-Chat-v1.0`
- System Prompt: Roughly 1500 tokens (adopted from OSTEP MLFQ scheduling chapter [8])
- Queries: 100 queries, each of length 10-20 tokens
- Max Output Length: 80 tokens
- Techniques Compared: vLLM, SGLang, swiftLLM [1]

**Algorithm: Workload Generation at Target QPS**

**Require:** LLM model `llm`, list of `prompts`, parameters `sampling_params`, target QPS `qps`
**Ensure:** Latencies of requests

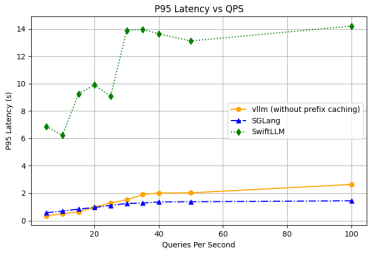
Initialize empty list `latencies`
`interval` $\leftarrow 1/$`qps`
`start_time` $\leftarrow$ current time
$i \leftarrow 0$
Initialize empty `backlog`
`next_time` $\leftarrow$ current time
**while** $i <$ length(`prompts`) **do**
    `now` $\leftarrow$ current time
    `intervals_passed` $\leftarrow \lfloor$(`now` $-$ `next_time`)$/$`interval`$\rfloor + 1$
                                ▷ Add missed prompts to backlog if multiple intervals passed
    **for** $k = 1$ **to** `intervals_passed` **do**

[1] SwiftLLM is a smaller version of vLLM, the codebase spans just 2000 lines, and is quite simpler and easier to understand [5]

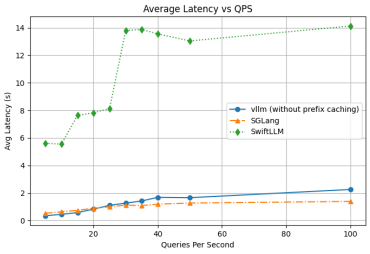
```
        if i < length(prompts) then
            Append prompts[i] to backlog
            i ← i + 1
        end if
    end for
    if backlog is not empty then
        batch_start ← current time
        llm.generate(backlog, sampling_params)
        batch_latency ← current time − batch_start  ▷ All requests in backlog share the same latency
        for all request in backlog do
            Append batch_latency to latencies
        end for
        Clear backlog
    end if
    next_time ← next_time + intervals_passed × interval ▷ Sleep if ahead of schedule
    sleep_time ← max(0, next_time − current time)
    sleep(sleep_time)
end while
return latencies
```

We primarily measure the following metrics:

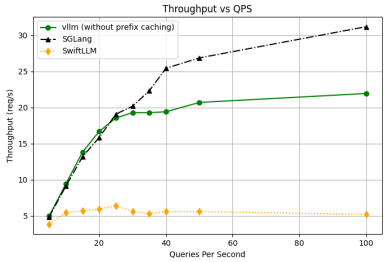
- **Throughput (QPS)**: Total number of requests / Total time taken
- **Average Latency (s)**: Average time taken per request, basically the time a user will have to wait
- **P95 Latency (s)**: 95th percentile latency across all requests, this helps us understand the tail latency, which in turn blocks the next batch of requests




(a) P95 Latency




(b) Average Latency




(c) Throughput

Figure 7.1: Performance metrics across varying QPS

- The throughput for swiftLLM is really low as compared to vLLM and SGLang, due to ineffective memory management and lack of batching optimizations.
- vLLM v0 did not support automatic prefix caching, leading to higher latencies.
- The throughput numbers flatten out for vLLM and SGLang at higher QPS due to GPU compute saturation.
- Overall, with radix tree based prefix caching, SGLang achieves the best performance across all metrics.

## 7.2 GPU Memory Breakdown

To further understand the memory behavior of these inference workloads, we profile the GPU memory allocation of vLLM during execution, isolating contributions from model weights, activations, and KV cache components.

For the TinyLlama-1.1B model, here is the breakdown of GPU memory usage during inference with vLLM, this helps us understand how much memory is occupied by different components, including the KV cache used for prefix caching.

| **Component** | **Meaning** | **Memory Occupied** |
|---|---|---|
| Weights | Model parameters of the neural network. Static during inference. | 0.93 GiB |
| Peak Activation | Temporary tensors generated during forward pass. Depends on batch size and sequence length. | 4.07 GiB |
| Non-Torch | GPU memory used outside PyTorch tensors (framework/library overhead, buffers). | 0.02 GiB |
| CUDA Graph | Memory reserved for CUDA Graphs to optimize kernel execution and reduce CPU-GPU overhead. | 0.41 GiB |
| KV Cache: Intersection (Prefix) | Cached attention keys/values shared across requests (common prefix). | 1,280 tokens / 0.0157 GiB |
| KV Cache: Extras for Requests | Cached attention keys/values unique for each request. | 640 tokens (10 requests × 64 tokens) / 0.0078 GiB |
| KV Cache: Free | Remaining unused KV cache memory. | 2,672,880 tokens / 32.8445 GiB |

Table 7.1: GPU Memory Breakdown and KV Cache Usage

## 7.3 LLM Request Timelines

While throughput and latency provide aggregate measures, they conceal important scheduling dynamics. We therefore visualize the per-request timelines to analyze how prefill and decode phases are interleaved within vLLM's scheduling policy.

A request timeline shows when each request started (got into the waiting queue of the scheduler), when it's prefill got completed, and when the full output generation completed (token by token). This helps us visualize how well the system is able to batch request together and how the latencies vary across requests.

The following timelines are generated for different request rates (QPS). By default (without chunked prefill enabled), vLLM scheduler [7] prioritizes prefills and doesn't batch prefill and decode to the same batch. This policy optimizes the TTFT (time to thefirst token), but incurs slower ITL (inter token latency) and inefficient GPU utilization.

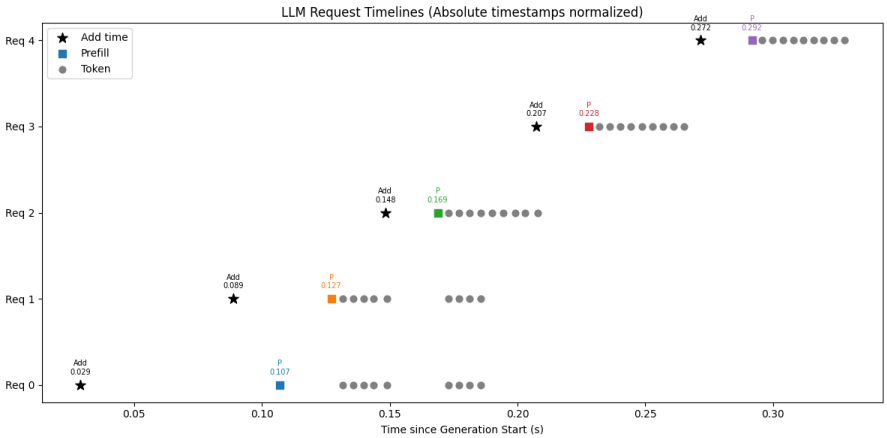



Figure 7.2: 20 Requests per second

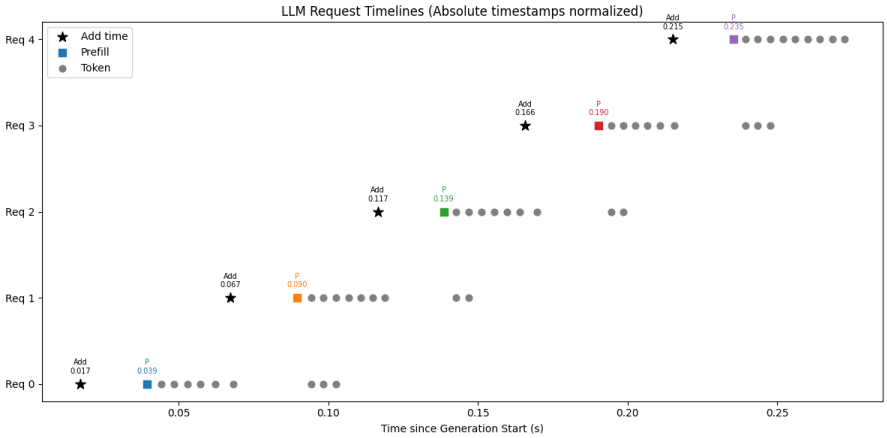



Figure 7.3: 25 Requests per second

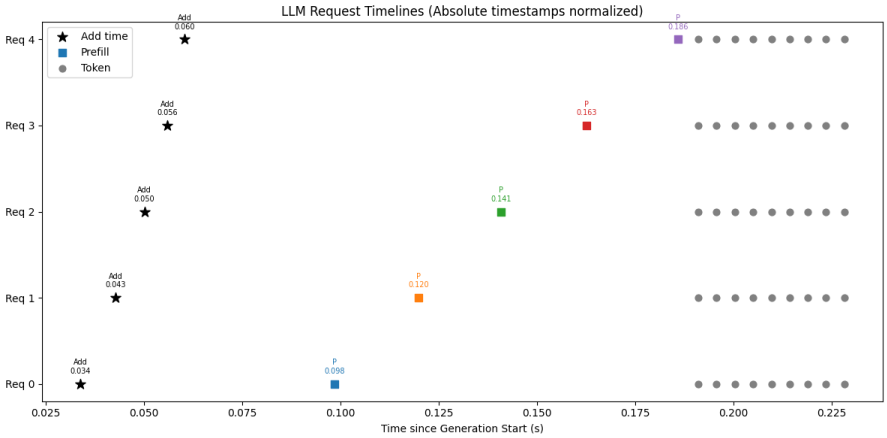



Figure 7.4: All requests at once

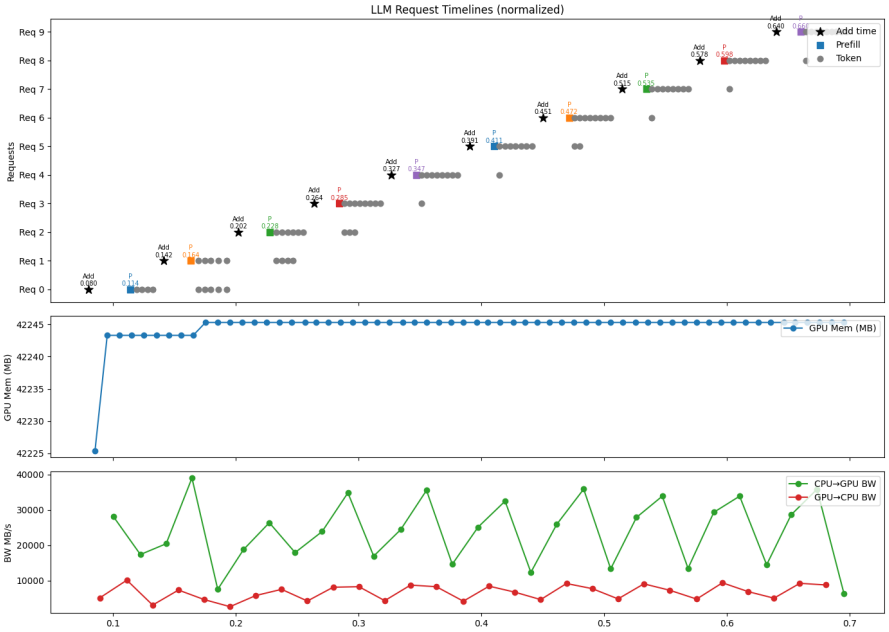



Figure 7.5: Request Timeline with GPU Metrics

As we can see in the plots above, there are stalls in decode stages of a request, because the scheduler prioritizes prefill stages of other requests.

### 7.3.1 Chunked Prefill Enabled

To mitigate the imbalance between prefill and decode prioritization, vLLM introduced a "chunked prefill" mechanism. We next study how this impacts latency and throughput under the same workload.

Once chunked prefill is enabled, the policy is changed to [7]:

- Prioritize decode requests: It batches all pending decode requests to the batch before scheduling any prefill.
- When there are available token budgets (`max_num_batched_tokens`), it schedules pending prefills. If the last pending prefill request cannot fit into `max_num_batched_tokens`, it chunks it.

This policy has two benefits:

- It improves ITL (inter token latency) and generation decode because decode requests are prioritized.
- It helps achieve better GPU utilization by locating compute-bound (prefill) and memory-bound (decode) requests to the same batch.

Smaller batch size achieves better ITL because there are fewer prefills interrupting decodes. Higher batch size achieves better TTFT as you can put more prefill to the batch.

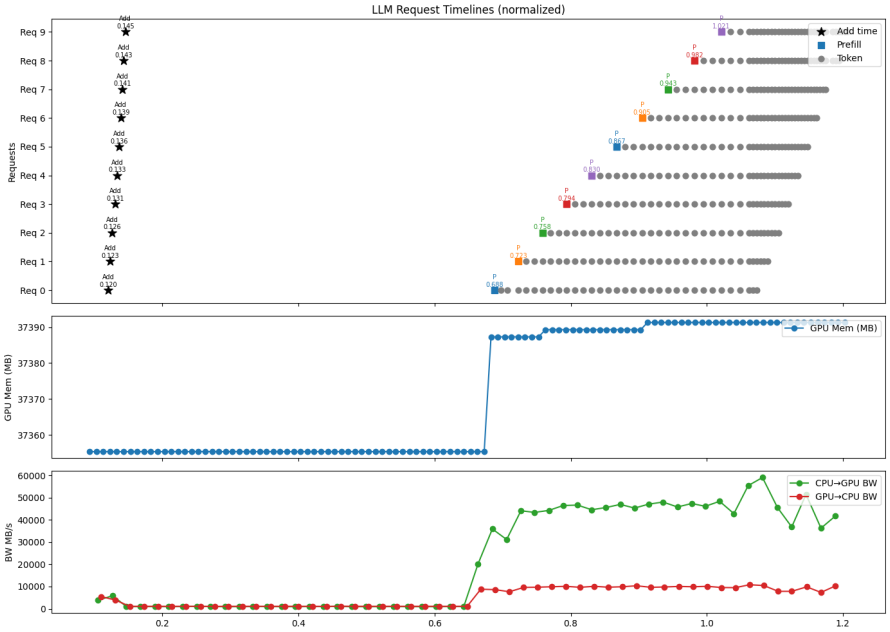



Figure 7.6: Chunked Prefill Enabled - All requests at once

## 7.4 Maximum Throughput with vLLM

We next quantify the maximum achievable throughput of vLLM under different scheduling policies to validate the observed trends.

We sent 100 requests all at once to vLLM server, and measured the number of prefills and deocodes for a single batch (batches are generated after every model run). This experiment helped us understand the scheduler algorithm of vLLM in a much better way.

When chunked prefill is enabled, we got a throughput of 26.04 QPS. The total time taken was 3.84s for 100 requests.

When chunked prefill is disabled, we got a throughput of 20.12 QPS. The total time taken was 4.97s for 100 requests.

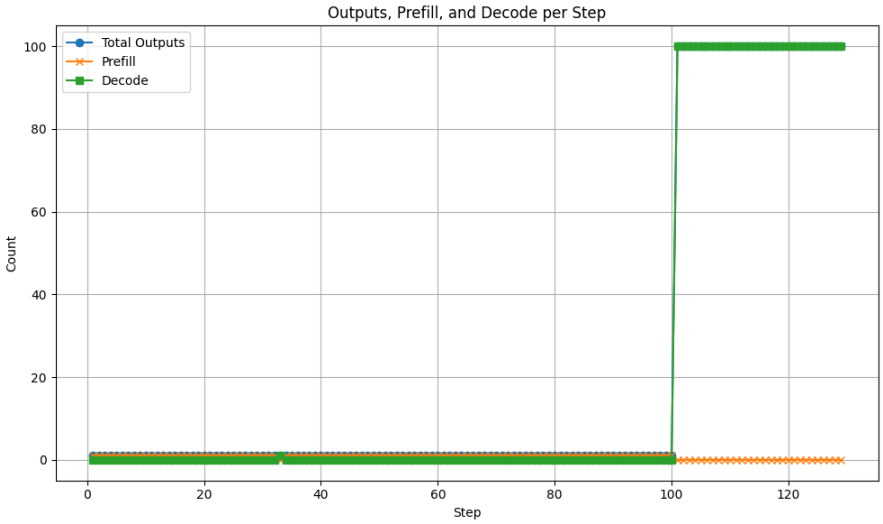



Figure 7.7: Number of batch outputs - chunked prefill disabled

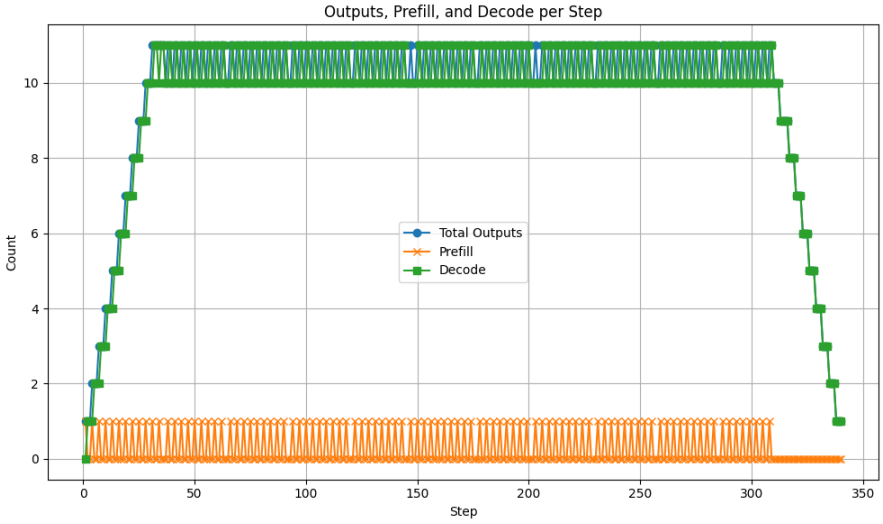



Figure 7.8: Number of batch outputs - chunked prefill enabled

As we can see from the above plots, when chunked prefill is disabled, we perform all the prefill operations in the start itself (because of large token sizes, and small number of max batched tokens, we can't afford more than one prefill in a single batch). After all the prefills are performed, we move to the decoding stage, where each new request just takes a size of one token, leading to a lot of decode operations in a single batch.

When chunked prefill is enabled, we are able to interleave prefill and decode operations better, leading to improved GPU utilization and throughput.

## 7.5 Throughput vs Load Comparison For vLLM

Having established baseline performance, we now compare throughput scalability across different optimization configurations—specifically, prefix caching and chunked prefill.

From now on, we enable automatic prefix caching in vLLM for better performance (switched from vLLM v0 to vLLM v1 engine).

In the following experiment, we compare the throughput in three cases:

- vLLM with prefix caching and chunked prefill enabled
- vLLM with just chunked prefill enabled
- vLLM with no optimizations

We vary the load (QPS) from 5 to 500, and measure the achieved throughput.

As we can see from the plot above, with prefix caching enabled, we are able to achieve much higher throughput at higher loads, for example the maximum we reached was around 235 QPS, whereas without prefix caching, the maximum we reached was just around 30 QPS.

This is because, our workload has a large common prefix, which no longer needs to be recomputed for every request, leading to significant speedups.

The improvement of chunked prefill for larger loads is negligible.

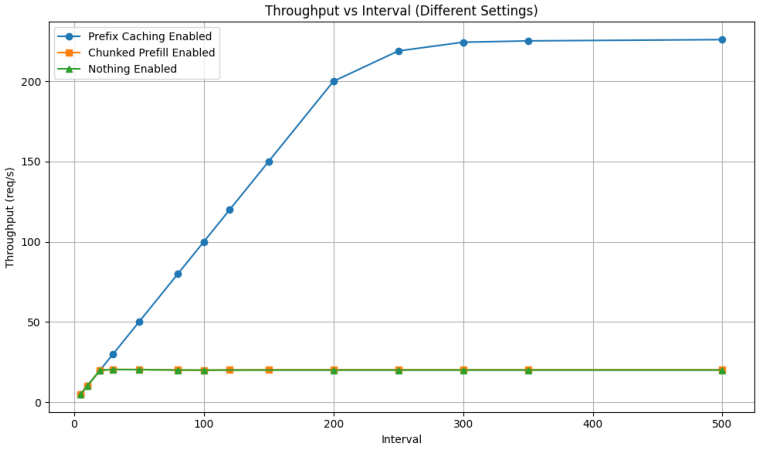



Figure 7.9: Throughput vs Load

Next, we compared different models (TinyLlama-1.1B, Llama-2-7B) for varying workloads (again with prefix caching and chunked prefill enabled). As we can see, larger models have lower throughput due to higher compute requirements. They have larger number of attention heads and layers, leading to more compute intensive operations.

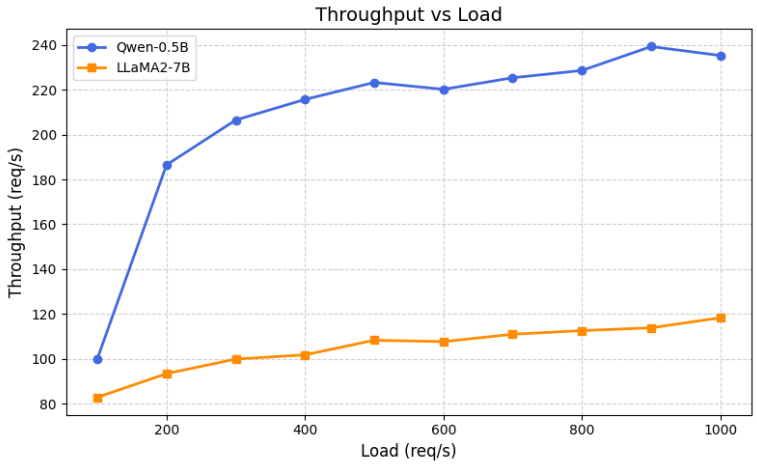



Figure 7.10: Throughput vs Load for different models for heavy workloads

## 7.6 Compute Utilization across various max batch sizes

To get a better idea of how well vLLM utilizes the GPU across differe max batch sizes (number of requests we can fit into a single scheduler batch), we measured two metrics - SM Activity and SM Occupancy through NVIDIA DCGM [4]. As we can see, for smaller models (such as Qwen 0.5B used in this case), the compute utilization stops increasing drastically after a batch size of 1000 requests. Increasing the batch size further doesn't help much, as the model is already able to saturate the GPU compute. However, for larger models, this threshold will be much lower (for example around 100 requests for Llama-2-7B).

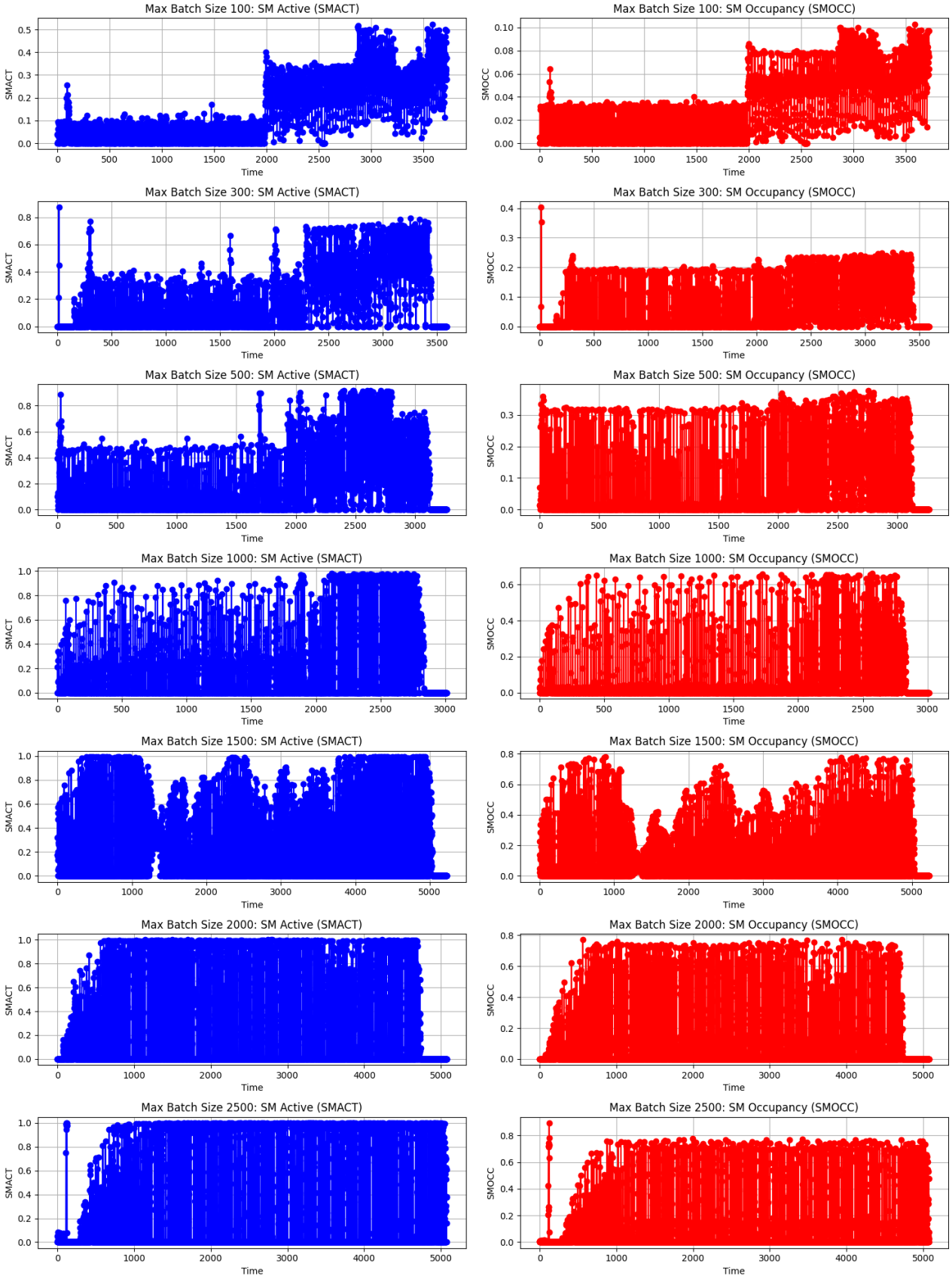



Figure 7.11: SM Active and SM Occupancy across different max batch sizes

# Chapter 8

# Experiments - Multiple Family of Requests

## 8.1 Introduction

While the previous chapter focused on workloads where all queries share a single long prefix, real-world applications often involve heterogeneous families of requests—each with its own distinct context or document. For instance, in an enterprise setting, different teams might query separate knowledge bases, or in an educational setup, multiple courses could simultaneously host LLM-based assistants trained on different lecture materials. These scenarios represent a multi-family inference workload, where each family maintains an independent prefix but shares the same model and hardware resources.

## 8.2 How to generate such a workload?

From the previous experiments, we already have a large system prompt of around 1500 tokens. Now, we need to perform experiments with multiple families of requests, where each family shares a large prefix in common (the system prompt).

One naive way to generate such a dataset, is to write multiple unique system prompts, each around 1500 tokens. However, for larger number of families, this becomes tedious and time-consuming.

A smart hack (Fig. 8.1) to generate multiple families of requests, while still sharing the same system prompt, is to prepend a unique identifier (like "AA", "AB", "AC", etc.) to the existing system prompt. This way, each family has a unique prefix, but the majority of the content (the system prompt) remains the same across all families. Since, we do not care about the response quality in these experiments (for now), this approach is sufficient to simulate multiple families of requests with a large shared prefix.

Also, since the starting identifiers are unique for each family, the block hashes will be different, leading to separate KV Cache blocks for each family, despite most of the content being identical.

## 8.3 Families of Requests across various models

In our first experiment, we compare the throughput of handling multiple families of requests versus a single family with the same total number of requests. We use 10 families, each with

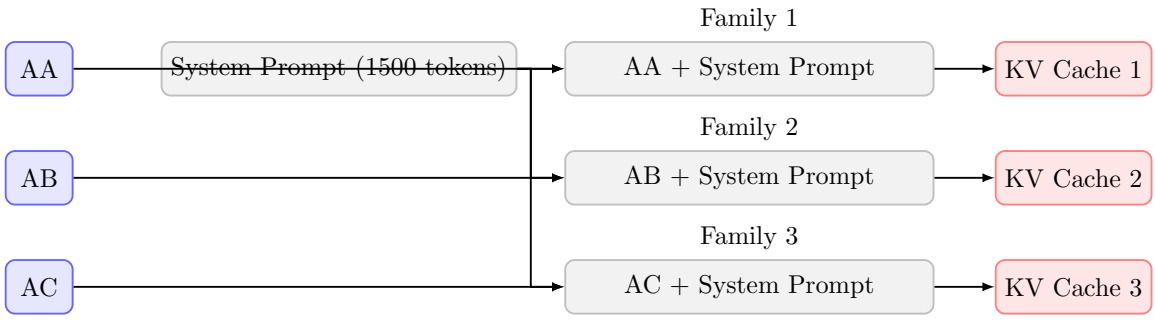



Figure 8.1: Generation of families by prepending unique identifiers, resulting in separate KV Cache blocks despite shared system prompt content.

1000 requests, and compare it to a single family with 10000 requests. The models chosen for this experiment are Qwen 0.5B, Llama 7B and LongChat 13B. Here are the results:

| **Model Chosen** | **10 families, each with 1000 queries** | **1 family with 10000 requests** |
|---|---|---|
| Qwen 0.5B | 294.228 req/s | 308.750 req/s |
| Llama 7B | 43.923 req/s | 124.347 req/s |
| LongChat 13B | 22.415 req/s | 70.097 req/s |

Table 8.1: Throughput with multiple families across various models

As we can see, as the model size increases, the throughput degradation when handling multiple families becomes more pronounced. This is likely due to the increased attention computation overhead from having to manage multiple KV Cache blocks for different families, which becomes more significant with larger models.

## 8.4 Effect of Batch Decomposition on Inter-Scheduler Latency

The observed throughput differences raise questions about why multi-family scenarios degrade performance so sharply. To better understand the internal scheduling behavior, we decompose the inference process into its constituent stages—prefix prefill, unique prefill, and decode—and examine their individual contributions to end-to-end latency.

We divide a query into 3 components - prefix prefill (prefill of the large shared prefix), prefill (of unique suffix/query) and decode (generation of tokens). We vary the batch decomposition across these 3 components and measure the time spent in each component (basically the difference of time instants between two calls to the scheduler).

As we can see from the figure, the time spent in prefix prefill is significantly higher than the other two components, which is expected since the prefix is large (around 1500 tokens). Also, the overall time spent is heavily influenced by the number of prefix prefills we have in that batch. It is more or less independent of the number of suffix prefills and decodes.

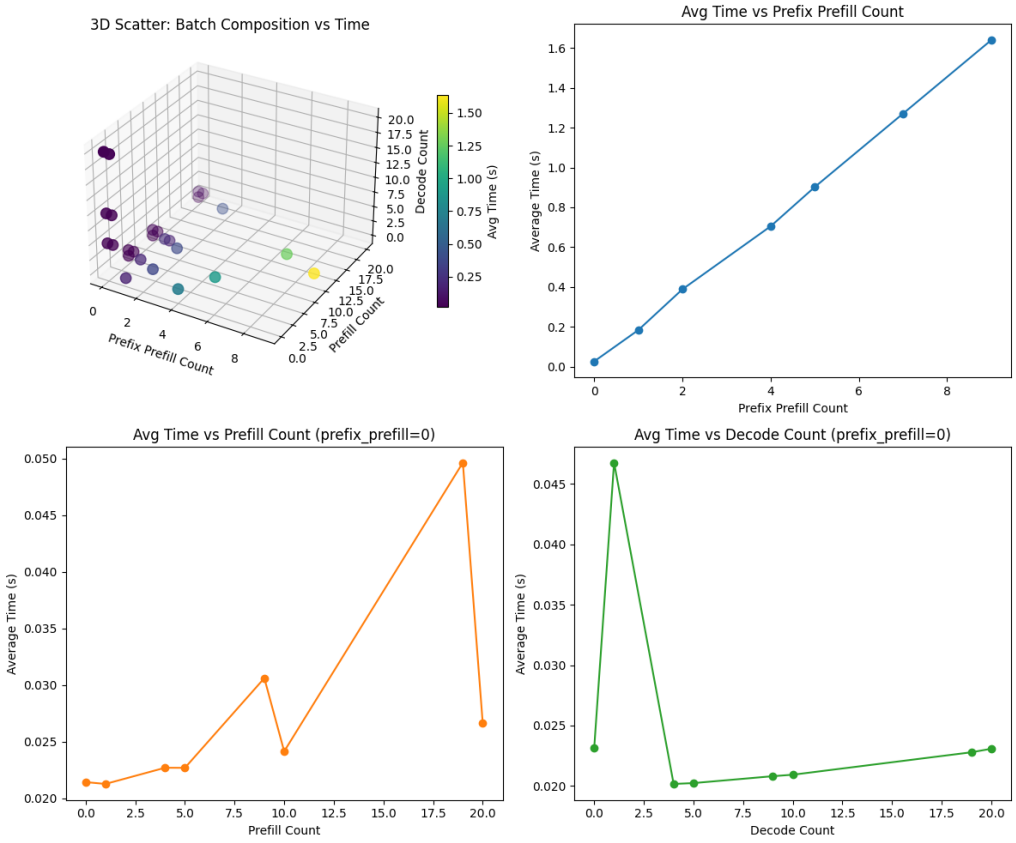



Figure 8.2: Times spent across different batch decompositions

## 8.5 Merging queries belonging to the same family into a single batch

Through a few more trials and experiments, we found that merging queries from the same family into a single batch (basically making the batch homogeneous with respect to family IDs) leads to significant throughput improvements.

This is likely because of the fact that each family shares a large number of KV Cache blocks, so if multiple queries from the same family fit into a single batch, we will have to load a large number of KV Cache blocks from the HBM into the L2 shared cache. Since, the size of an L2 shared cache is limited, having a heterogeneous batch with queries from different families will lead to cache thrashing and evictions, leading to more HBM loads and lower throughput.

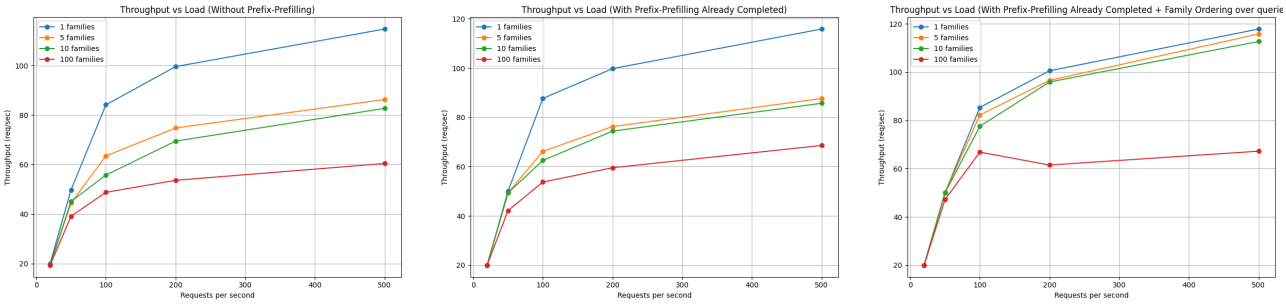
However, when we merge queries from the same family into a single batch, we reduce the number of unique KV Cache blocks needed for that batch, leading to better cache utilization and higher throughput.

Naively, we can sort the incoming queries based on their family IDs before forming batches. This way, queries from the same family are more likely to be grouped together in a batch.

| Number of Families | Number of queries in each family | Max Batch Size | Ordered by Family ID | Random Permutation |
|---|---|---|---|---|
| 10 | 10 | 10 | 12.28 | 11.981 |
| 10 | 100 | 100 | 59.601 | 55.704 |
| 10 | 100 | 200 | 61.693 | 60.923 |
| 100 | 100 | 100 | 42.882 | 39.172 |

Table 8.2: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

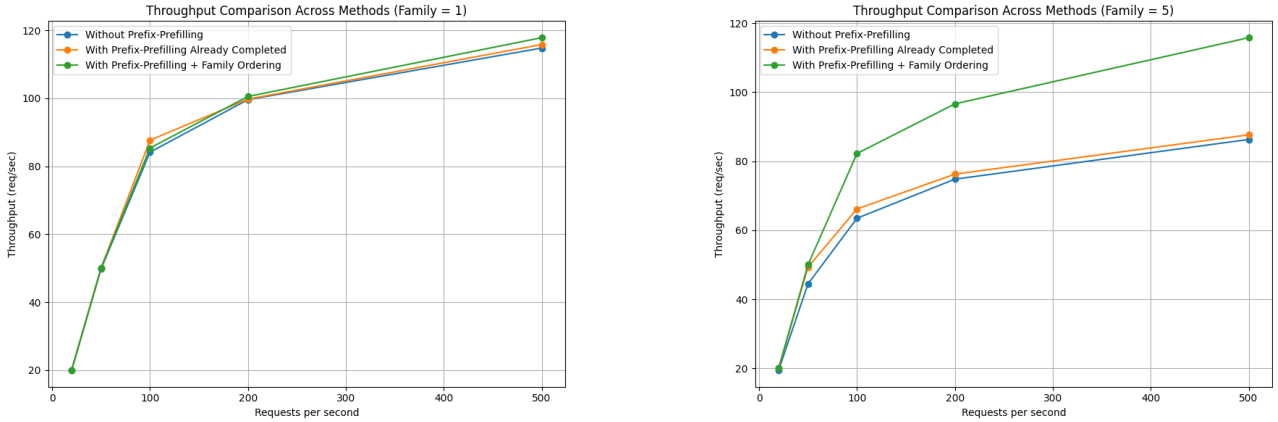
For Llama3.1-8B, here are the plots for varying workloads and number of families:




(a) Without prefix pre-filling (b) With prefix pre-filling (c) Ordered by family IDs

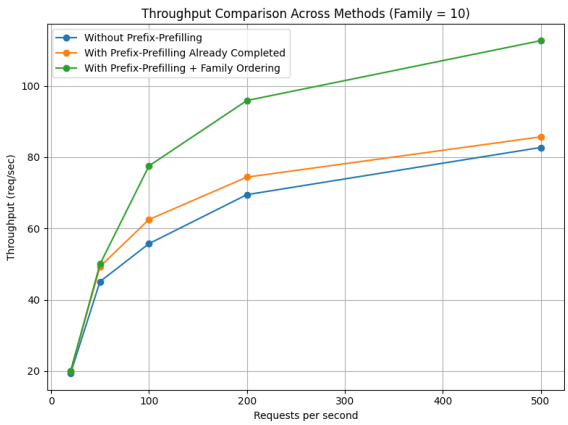
Figure 8.3: Throughput vs Load across different pre-filling strategies

Now, as we can see from the second image, where all the system prompts were passed in the model beforehand (prefix pre-filling), which ensured that all the KV Cache blocks were already in the HBM, there was still a significant difference between the throughput numbers across different number of families. This suggests us that the main bottleneck is not caused by the number of computations to generate the KV Cache blocks, rather the memory bandwidth to load these blocks from HBM to L2 cache. Since, the working set size increases with number of families, the memory bandwidth becomes the bottleneck. However, as seen in the third image, when we order the incoming queries based on their family IDs, we get a significant throughput boost, because of better cache locality and reduced cache thrashing. Here are the plots for varying number of families (showing the improvement from ordering by family IDs):

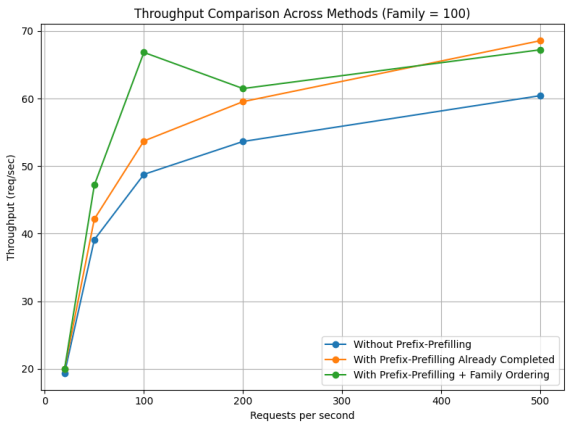



(a) Throughput vs Load for 1 family (b) Throughput vs Load for 5 families

Figure 8.4: Throughput vs Load across different numbers of families




(a) Throughput vs Load for 10 families




(b) Throughput vs Load for 100 families

Figure 8.5: Throughput vs Load across different numbers of families

## 8.6 Effect of Max Batch Size on Throughput for multiple families

All of these experiments are actually run with large number of requests, so that we get the best averaged throughput numbers. However, initially there is a small CPU overhead, to tokenize the requests and prepare them to be passed to the scheduler. This does not allow us to saturate the GPU in the starting phase. If we tokenize and store all the requests beforehand, and then pass them to the scheduler in a loop, we can saturate the GPU even for smaller loads.

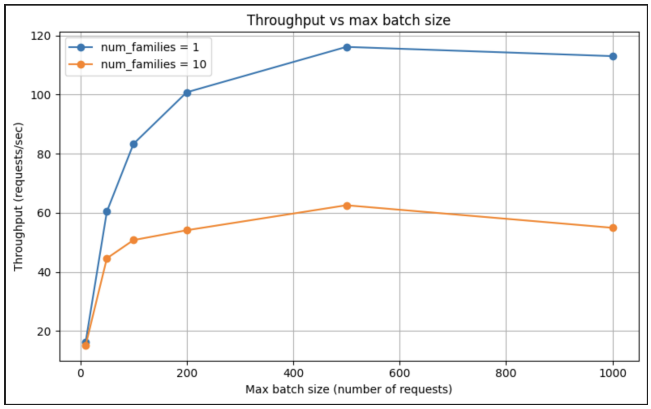



Figure 8.6: Throughput vs Max Batch Size for multiple families

## 8.7 Varying all the parameters together

So far, we have analyzed the influence of batch size and family composition independently. However, in practice, these factors interact—along with workload intensity—to determine overall performance. We therefore conduct a comprehensive study varying all parameters jointly to capture the combined dynamics of multi-family inference. So, we primarily have the following parameters:

- Number of Families
- Workload - requests per second
- Max Batch Size
- The nature of the batch - whether ordered by family IDs or random permutation:
  - 0: Ordered by family IDs

– 1: We make chunks of size 50 from the sorted list and permute each chunk randomly (allows us to have some level of prefix sharing between consecutive queries)
– 2: Completely random permutation

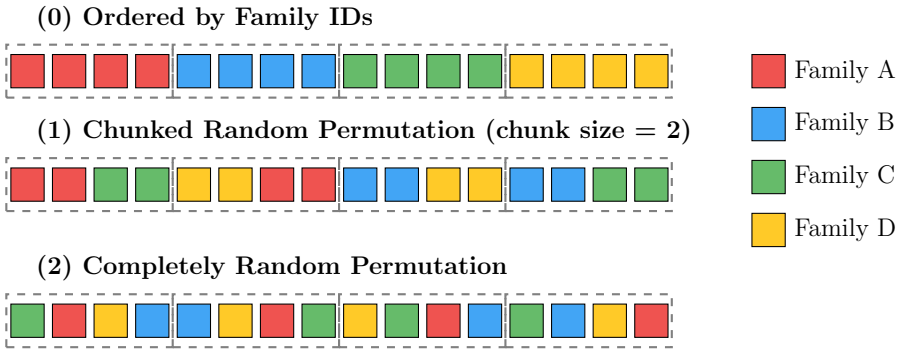



Figure 8.7: Visualization of query ordering strategies for 4 families (4 requests each): (0) sorted, (1) chunked random, (2) completely random. Each dashed box represents a batch of 4 requests.

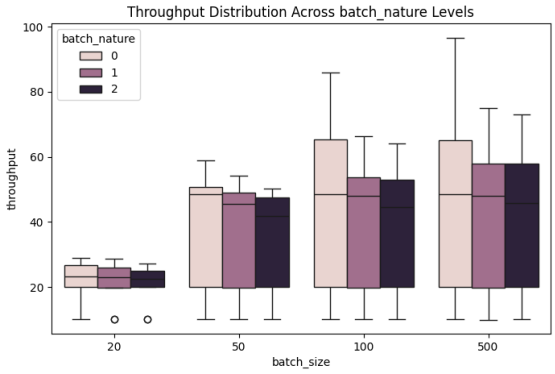



Figure 8.8: Throughput difference across different batch nature levels

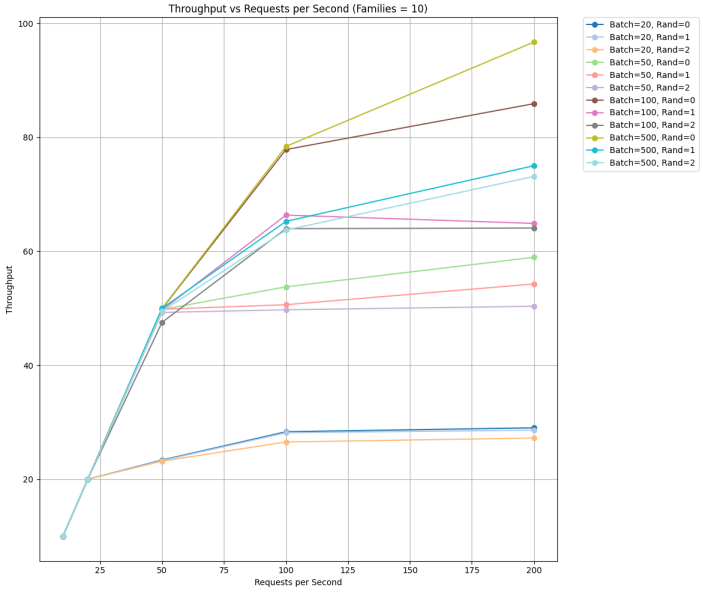



Figure 8.9: Throughput vs Workload by varying other parameters (10 families)

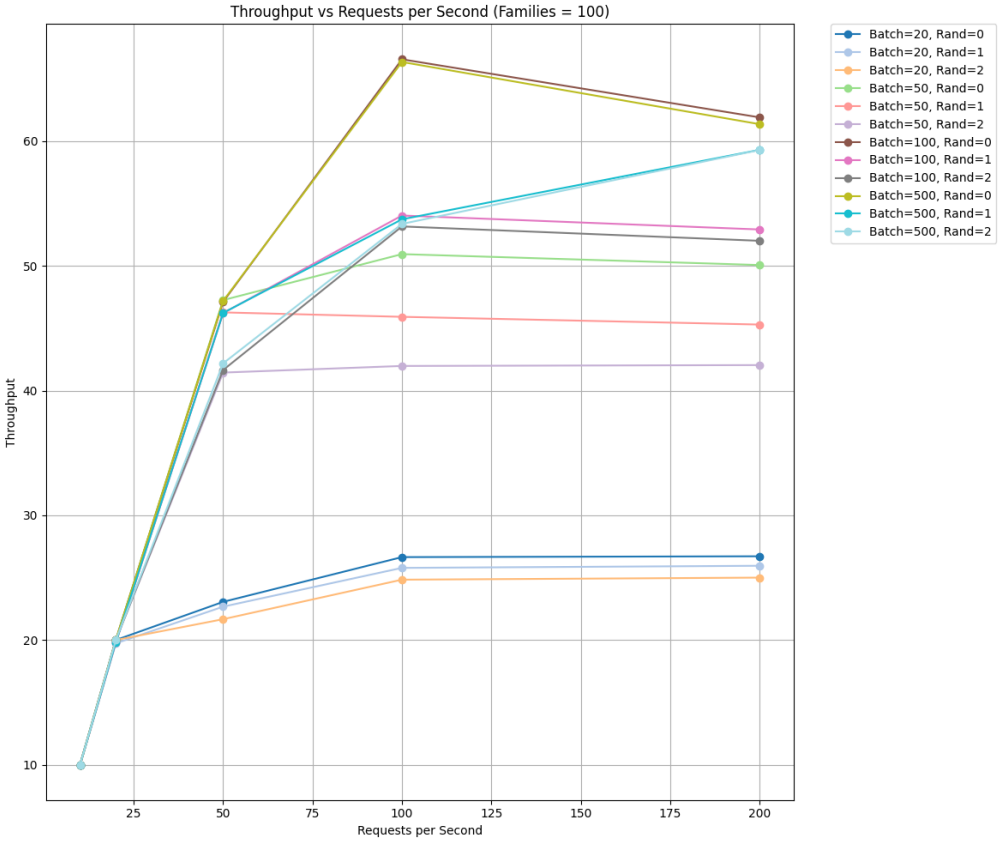



Figure 8.10: Throughput vs Workload by varying other parameters (100 families)

- As we can see, increasing the max batch size generally leads to higher throughput, as it allows better GPU utilization.
- Ordering the batches by family IDs (batch nature level 0) consistently provides the best throughput, due to improved cache locality and reduced memory bandwidth pressure.
- A batch nature level of 1 is roughly similar to level 2, which suggests that even a small amount of mixing in the batch (basically making the batch heterogeneous with $> 1$ number of families) leads to significant throughput degradation.
- This can be justified by the fact that, all the SMs share an L2 cache of limited size, so even a small increase in the working set size (number of unique KV cache blocks needed) can lead to cache thrashing and evictions, leading to more HBM loads and lower throughput.
- Often a homogeneous batch of smaller size performs significantly better than a larger heterogeneous batch (for example, check batch sizes of 100 and 500 in the plots above). This essentially suggests, that if we have a heterogeneous batch of 500, with 5 families inside it, we can possibly divide it into smaller batches of size 100, and achieve a much higher throughput.

The above results gave rise to another experiment. Let's say we have a parameter $k$. Now, before passing the requests to the model, we make chunks of size $k$, and permute each chunk randomly (for the above plots a batch nature level of 1 essentially meant a $k$ value of 50). We fix the workload at 500 req/sec, the batch size at 500, and the number of families at 10. The table on right shows how the throughput varies across $k$. Now, as we can see a $k$ value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family, however the throughput is much lower than a $k$ value of 500. This basically signals us that even one request of a different family will cause L2 cache thrashing.

Table 8.3: Throughput across different $k$ levels

| $k$ | **Throughput (req/sec)** |
|---|---|
| 200 | 89.35 |
| 300 | 87.49 |
| 400 | 86.50 |
| 499 | 88.19 |
| 500 | 112.66 |
| 1000 | 112.41 |

NCU (NVIDIA's Nsight Compute CLI) provides the option to measure the L2 cache hit rate, but it reports this metric separately for each kernel launched by the engine. Since vLLM launches thousands of kernels, collecting and aggregating this metric becomes impractical. To support our findings, we instead use another relevant metric, "DRAMA", which measures the fraction of cycles during which data is transferred to or from device memory. As shown in Figure 8.11, there is a clear difference between the two settings: for $k = 500$, the average DRAMA value is 0.18, while for $k = 499$ it increases to 0.22.

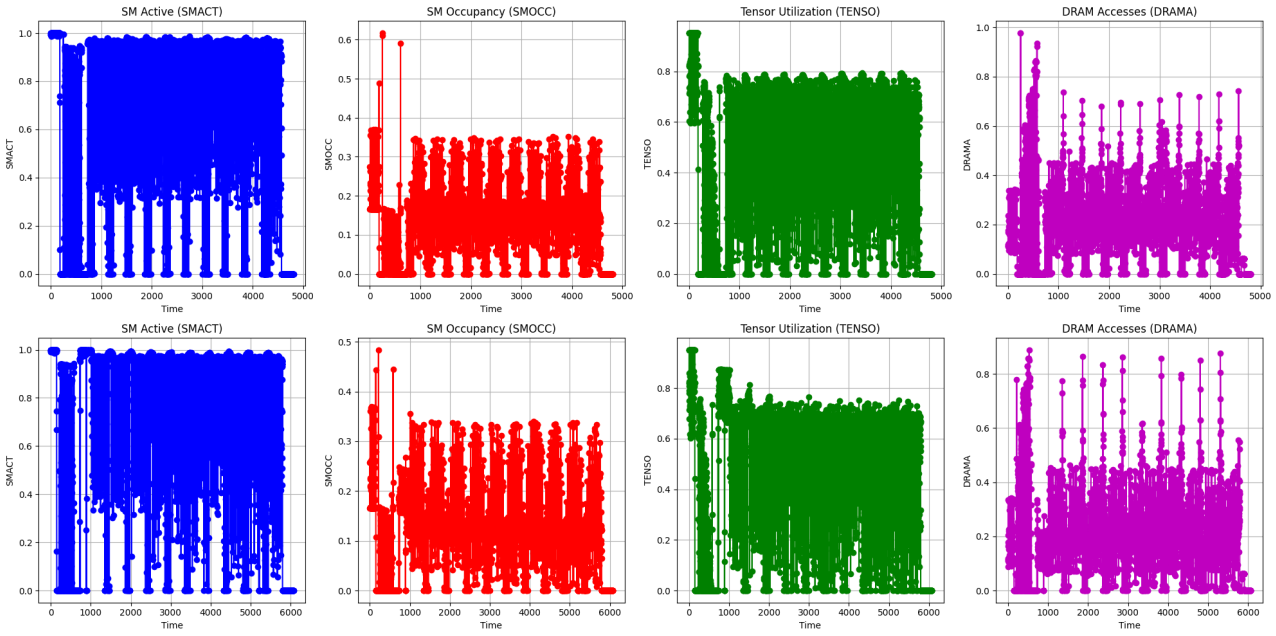



Figure 8.11: Comparison for $k = 500$ and $k = 499$

## 8.8 Comparison of vLLM with SGLANG - for multiple families

SGLANG suits better for structured multi-turn scenarios, and it has timed KV Cache eviction policy, thus it performs worse as compared to vLLM for such batch-like multi-family

scenarios:

| Number of Families | Batch Size | vLLM | vLLM (Family Ordered) | SGLANG | SGLANG (Family Ordered) |
|---|---|---|---|---|---|
| 10 | 10 | 15.0942 | 15.5041 | 13.3730 | 13.8388 |
| 20 | 100 | 51.1423 | 54.7801 | 57.5661 | 62.3397 |
| 50 | 50 | 40.5825 | 44.5922 | 30.2049 | 37.0809 |

Table 8.4: Throughput comparison (in req/s).

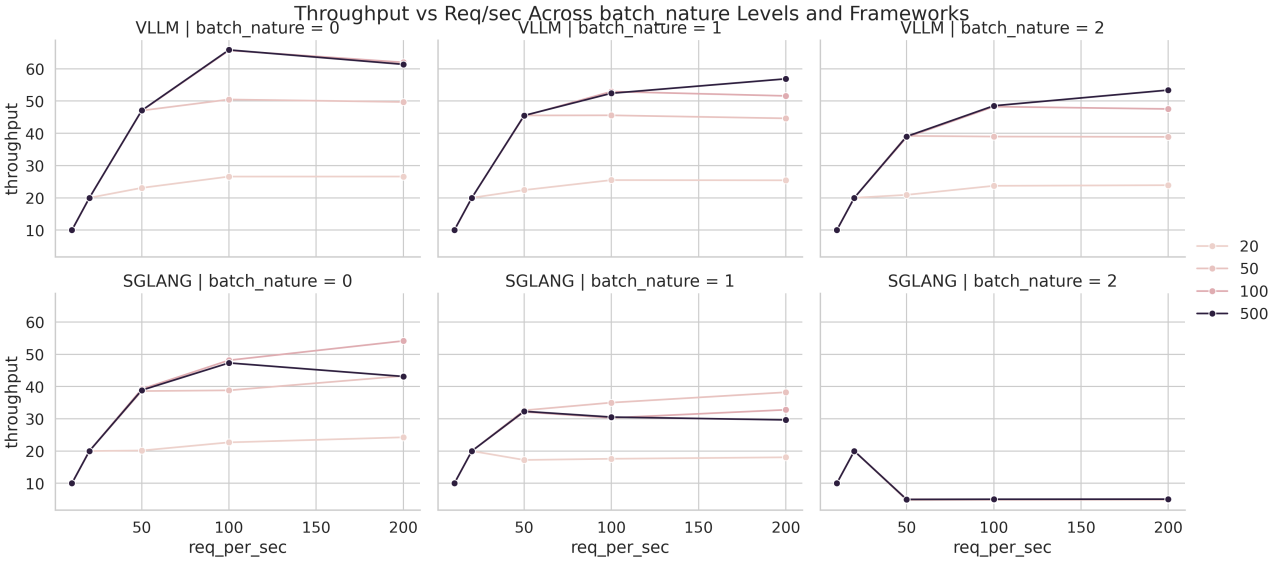



Figure 8.12: vLLM vs SGLANG - varying parameters

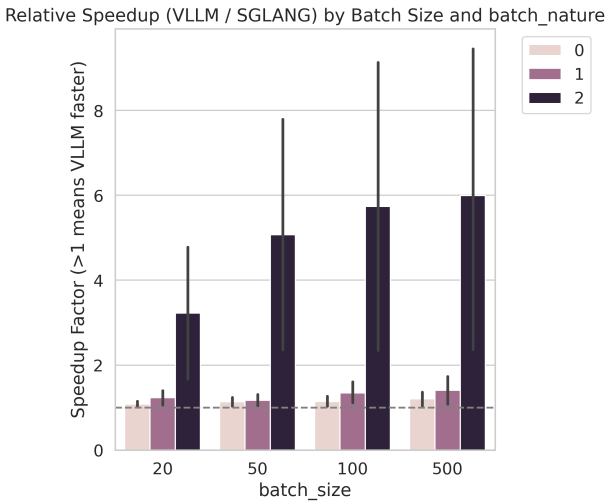



Figure 8.13: vLLM vs SGLANG - Relative Speedup across randomness levels

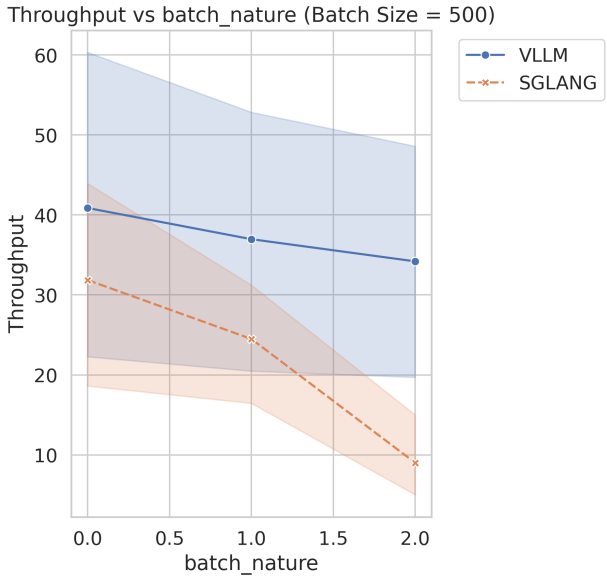



Figure 8.14: vLLM vs SGLANG - Throughput levels for a batch size of 500

## 8.9 GPU Utilization Metrics across different family and batch sizes

We also analyzed compute utilzation across varying batch sizes and number of families - just to get the confidence that our GPU is being fully utilized. We used 3 metrics - SM Activity, SM Occupancy and Tensor Core Utilization. It turns out that for larger model sizes, the number of warps are lesser (as each warp needs more memory), and thus the overall SM Occupancy numbers are much lower.

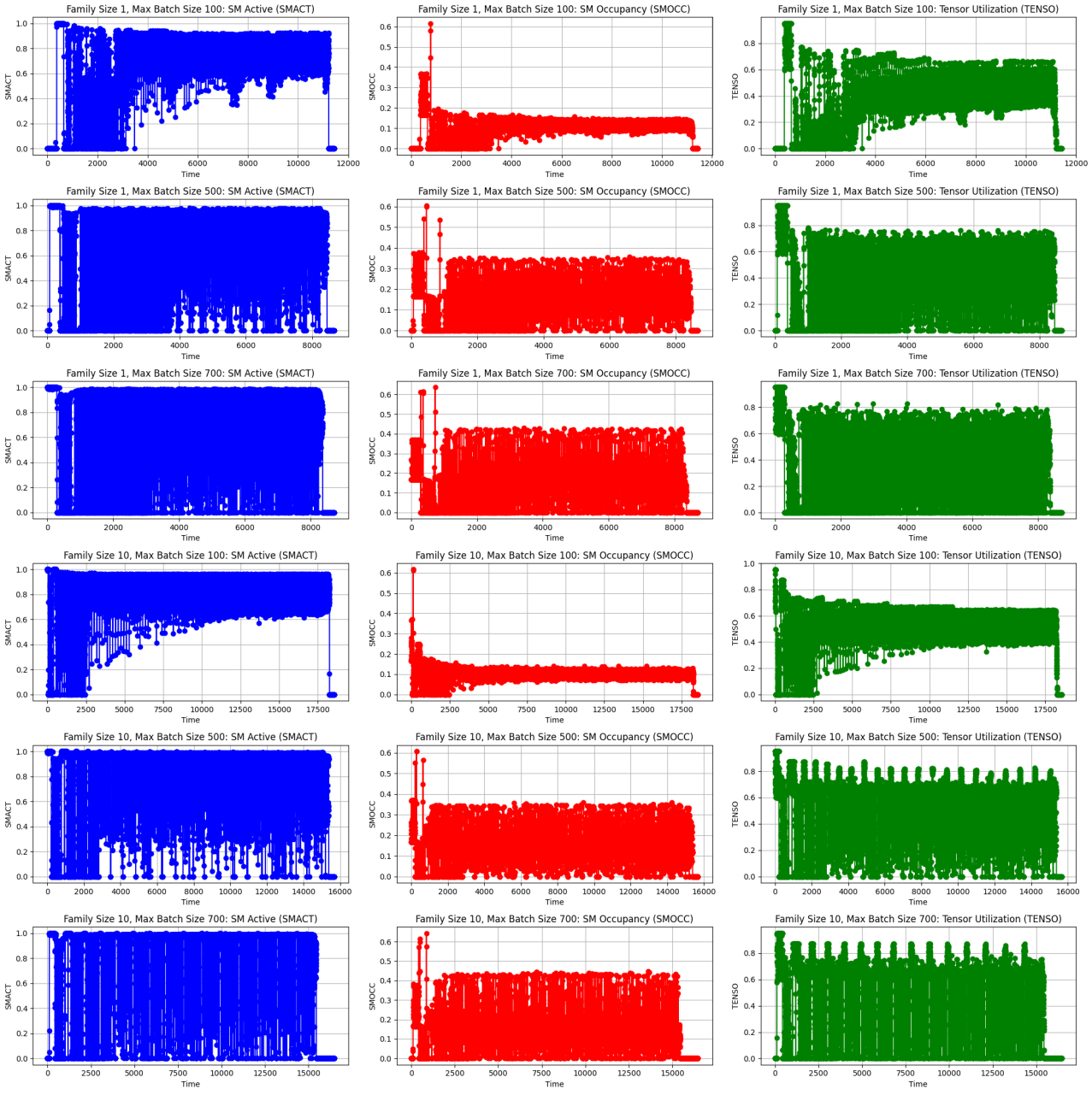



Figure 8.15: SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes and max batch sizes

# Chapter 9

# Future Steps

We outline below several directions for future work that we plan to pursue in the coming months:

- **Improved Scheduler:** Experiments across multiple families indicate that a larger batch size is not always optimal—especially when the batch is heterogeneous. A potential improvement is to design a scheduler that takes into account factors such as the L2 cache size, the working set size of the current batch, the number and size of KV Cache blocks being targeted, and dynamically decides whether forming smaller, more homogeneous batches can outperform larger, mixed ones.

- **SM-level Optimization:** In addition to the shared L2 cache, each Streaming Multiprocessor (SM) has a smaller L1 cache shared among its warps. To increase the L1 cache hit rate, it is desirable to schedule requests from the same family on nearby (or the same) SMs. The CUDA runtime scheduler typically follows a least-loaded policy—assigning kernels to the least occupied SM. However, this can lead to cache thrashing when requests from different families share the same SM. One possible approach is to assign each attention kernel an identifier derived from its family ID and preferentially schedule it on an SM already hosting requests from that family.

- **KV Cache Offloading:** As the number of families grows, the GPU's HBM may eventually become insufficient to hold all KV Cache blocks. In such scenarios, certain blocks must be offloaded to the CPU. This necessitates the design of an efficient eviction policy, ideally guided by the degree of prefix sharing among the currently scheduled batches.

- **Response Quality:** Thus far, our focus has been primarily on inference performance. However, another promising direction is to improve response quality for requests sharing a large common context. One approach is to fine-tune the model specifically for such shared contexts and compare its performance with that of a baseline model that receives the shared context as part of a system prompt.

# Appendix A

# Maths behind LLMs

## A.1 Transformer FLOPs

The motivation for this section comes from [9]. Counting the number of floating-point operations (FLOPs) in Transformers is a useful way to estimate compute requirements and measure efficiency.

### A.1.1 FLOPs Calculation

Let's explain each operation and variable here:

- Embed: learned token embeddings and learned positional embeddings. $d_{model}$ is the dimensionality of the residual stream.
- Attention: QKV: linear layer in multi-head self-attention to project input into queries, keys, and values. $n_{layers}$ is the number of layers. $d_{attn}$ is the dimensionality of the output of multi-headed attention which is equal to $d_{key} \times n_{heads}$. $d_{key}$ is the dimension of the key, query, and value projections. $n_{heads}$ is the number of attention heads in a layer. In practice, Transformers are implemented such that $d_{attn} = d_{model}$.
- Attention: Mask: dot-product between query and keys. $n_{ctx}$ is the context/sequence length.
- Attention: Project: linear layer to project concatenated attention heads output to $d_{model}$.
- Feedforward: two linear layers in the MLP block. $d_{ff}$ is the size of the output dimensionality of the first linear layer. $d_{ff} = 4d_{model}$ is commonly used.
- De-embed: linear layer to obtain logits over vocabulary. $n_{vocab}$ is the number of tokens in the vocabulary.

Combining all the non-embedding FLOPs terms gets us

$$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn}$$

| Operation | Parameters | FLOPs per Token |
|---|---|---|
| Embed | $(n_{\text{vocab}} + n_{\text{ctx}})d_{\text{model}}$ | $4d_{\text{model}}$ |
| Attention: QKV | $n_{\text{layer}}d_{\text{model}}3d_{\text{attn}}$ | $2n_{\text{layer}}d_{\text{model}}3d_{\text{attn}}$ |
| Attention: Mask | — | $2n_{\text{layer}}n_{\text{ctx}}d_{\text{attn}}$ |
| Attention: Project | $n_{\text{layer}}d_{\text{attn}}d_{\text{model}}$ | $2n_{\text{layer}}d_{\text{attn}}d_{\text{model}}$ |
| Feedforward | $n_{\text{layer}}2d_{\text{model}}d_{\text{ff}}$ | $2n_{\text{layer}}2d_{\text{model}}d_{\text{ff}}$ |
| De-embed | — | $2d_{\text{model}}n_{\text{vocab}}$ |
| **Total (Non-Embedding)** | $N = 2d_{\text{model}}n_{\text{layer}}(2d_{\text{attn}} + d_{\text{ff}})$ | $C_{\text{forward}} = 2N + 2n_{\text{layer}}n_{\text{ctx}}d_{\text{attn}}$ |

Figure A.1: FLOPs Calculation Breakdown [9]

### A.1.2 Model parameters (TinyLlama-1.1B-Chat)

$$
\begin{aligned}
L &= 22 \quad \text{(number of transformer layers)} \\
n_{\text{heads}} &= 32 \\
d_{\text{ff}} &= 5632 \quad \text{(intermediate / FFN size)} \\
V &= 32000 \quad \text{(vocabulary size)} \\
n_{\text{ctx}} &= 2048 \quad \text{(context / sequence length)}
\end{aligned}
$$

### A.1.3 FLOPs Calculation for TinyLlama-1.1B-Chat

$$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn} = 2.2 \times 10^9 + 1.84549376 \times 10^8 = 2.3845 \times 10^9 \text{ FLOPs.}$$

### A.1.4 Efficiency Calculation

We ran inference on TinyLlama-1.1B-Chat using a single Nvidia RTX6000 Ada GPU. Based on the docs from Nvidia [2], the theoretical peak performance for FP16 is 1457 TFLOPs.

### A.1.5 Efficiency Calculation

We ran inference on TinyLlama-1.1B-Chat using a single NVIDIA RTX6000 Ada GPU. Based on the docs from NVIDIA [2], the theoretical peak performance for FP16 is 1457 TFLOPs.

So, the time taken for a forward pass of $n_{ctx} = 2048$ tokens should be:

$$T = \frac{C_{forward} \times n_{ctx}}{\text{Peak TFLOPs}}$$

$$= \frac{(2.3845 \times 10^9) \times 2048}{1.457 \times 10^{15}}$$

$$= \frac{4.879 \times 10^{12}}{1.457 \times 10^{15}}$$

$$= 3.35 \times 10^{-3} \text{ seconds } (\approx 3.35 \text{ ms})$$

During our experiments, we observed that the actual time taken for a forward pass of 100 prompts, each with a context length of 2048 tokens, was approximately 4.4s. Therefore, the time taken for a single forward pass with a context length of 2048 tokens is approximately 0.044s (or 44ms).

Thus, the efficiency can be calculated as:

$$\text{Efficiency} = \frac{\text{Theoretical Time}}{\text{Actual Time}} = \frac{3.35 \text{ ms}}{44 \text{ ms}} \approx 7.6\%$$

### A.1.6 Summary of the nomenclature used

| Variable | Meaning |
|---|---|
| $d_{model}$ | Dimensionality of the residual stream (hidden size of the Transformer). |
| $L$ or $n_{layers}$ | Number of Transformer layers (blocks). |
| $n_{heads}$ | Number of attention heads in each multi-head attention layer. |
| $d_{key}$ | Dimensionality of each head's key, query, and value vectors. |
| $d_{attn}$ | Dimensionality of the concatenated multi-head attention output. Typically $d_{attn} = d_{key} \times n_{heads} = d_{model}$. |
| $n_{ctx}$ | Context length or sequence length, i.e., number of tokens processed in one forward pass. |
| $d_{ff}$ | Size of the hidden layer in the feedforward (MLP) block. Commonly $d_{ff} = 4d_{model}$. |
| $n_{vocab}$ or $V$ | Vocabulary size: number of tokens in the tokenizer vocabulary. |
| $N$ | Total number of trainable parameters in the model. |
| $C_{forward}$ | FLOPs required for a single forward pass per token. |

Table A.1: Variables used in FLOPs

# Appendix B

# Source Code

The source code for this project is available at the following GitHub repository:

https://github.com/sakshamrathi21/Structured_LLM_Optimization

The repository contains all the necessary code files, scripts, and documentation to reproduce the experiments and results presented in this report.

# Bibliography


[1] Understanding the workload characteristics of large language model development. URL https://www.usenix.org/publications/loginonline/understanding-workload-characteristics-large-language-model-development.

[2] Nvidia rtx 6000 ada generation. URL https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/rtx-6000/proviz-print-rtx6000-datasheet-web-2504660.pdf.

[3] How llms generate text for the rest of us. URL https://pm.dartus.fr/posts/2025/how-llm-generate-text/.

[4] Nvidia data center gpu manager (dcgm) user guide - metrics. URL https://docs.nvidia.com/datacenter/dcgm/latest/user-guide/feature-overview.html#metrics.

[5] swiftllm. https://github.com/interestingLSY/swiftLLM.

[6] vllm prefix caching. . URL https://docs.vllm.ai/en/stable/design/prefix_caching.html.

[7] vllm performance tuning. . URL https://docs.vllm.ai/en/v0.4.2/models/performance.html.

[8] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.10 edition, November 2023.

[9] A. Casson. Transformer flops. 2023. URL https://adamcasson.com/posts/transformer-flops.

[10] IBM. Sysml workshop. 2025.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[12] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Z. Yu, X. Tang, Y. Wang, and Y. Ding. Fasttree: Optimizing attention kernel and runtime for tree-structured llm inference. In *Proceedings of Machine Learning and Systems*, 2025.

[13] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL https://arxiv.org/abs/2312.07104.